

C + C++ + OOP + VC++

C/C++과 OOP(Object Oriented Programming)
그리고 Visual C++

Contents

❑ Contents

1. Foundation of Programming(C/C++)
2. OOP Concept
3. C++ is better than C
4. Foundation of Class
5. Advance of Class
6. Inheritance
7. Polymorphism
8. Programming
9. Windows Programming
10. Windows Programming- Sample
11. .NET Framework

1. Foundation of Programming(C/C++)

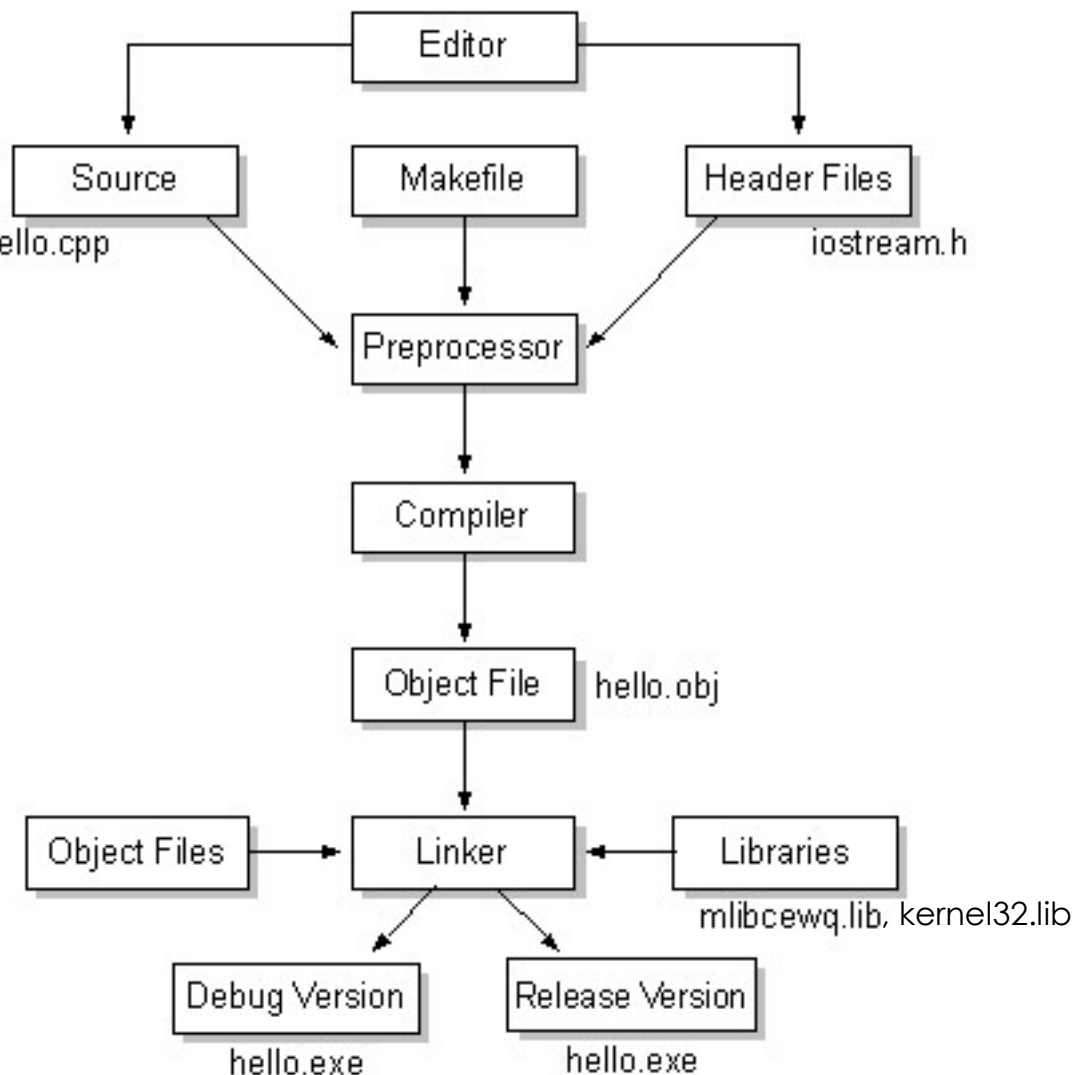
□ Contents

- 1) 프로그래밍의 기초
- 2) 프로그램의 구성요소
- 3) 변수와 상수
- 4) 연산자
- 5) 함수
- 6) 제어문
- 7) 입출력의 기초
- 8) 배열
- 9) 포인터
- 10) 문자와 문자열
- 11) 구조체
- 12) 변수의 범위

1) 프로그래밍의 기초

□ 프로그램 작성과정

1. Source File 작성:cp, cpp hello.cpp
2. Object File 작성
3. Linking
4. 실행파일 실행



1) 프로그래밍의 기초

Text Editor vs. IDE(Integrated Development Environment) vs. Visual Tools

Source Code vs. Execute Code

Native Code vs. P-Code

Interpreter vs. Compiler

Compile at Command Line

Borland C++: `bcc HelloWorld.cpp`

Borland C++ for Microsoft Windows: `bcc HelloWorld.cpp`

Borland Turbo C++: `tc HelloWorld.cpp`

Microsoft: `cl HelloWorld.cpp`

GCC: `g++ -o HelloWorld HelloWorld.cpp`

1) 프로그래밍의 기초

Microsoft Developer Studio 사용방법소개

실행 : 시작→프로그램→MS Visual Studio 6.0→MS Visual C++ 6.0

종료 : **File** →**Exit** 또는 **Close Button**

프로젝트생성 :

새 프로젝트 시작 : File →New... →Projects

프로젝트유형 : Win32 Console Application

프로젝트이름 : Hello

위치: Path\Hello

파일생성

프로젝트에 포함시킬 파일 지정

파일포함 : Project →Add to Project →Files...

파일이름 : Hello.cpp

빌드

Build →Build :프로젝트에 포함된 모든 파일을 다시 Compile, Link,
실행파일을 만든다.

1) 프로그래밍의 기초

실행

Build → Execute : Dos 창에 실행결과가 나타난다.

실행창 닫기

Close Button 또는 아무 키나 누른다.

프로젝트 닫기/열기

닫기: File → Close Workspace

열기: File → Open Workspace 에서 Hello.dsw를 연다.

도움말(MSDN: Microsoft Developer Network)

원하는 단어에 커서를 두고 F1을 누른다.

도움말 메뉴를 이용한다.

1) 프로그래밍의 기초

Error

Syntax Error :

Build결과 창에서 Error Message를 double Click해서 Error를 수정한다.

logic Error :

논리적으로 잘못된 결과가 발생 할때는 Debugging Tool을 이용한다.

Debugging

중단점 설정/해제 : F9

디버깅 시작: Build → Start Debug → go(F5)

디버깅 마침: Debug → Stop Debugging

내부추적 : Debug → Step Into (F11)

외부추적 : Debug → Step Over (F10)

추적탈출 : Debug → Step Out (Shift F11)

Warning vs. Error

1) 프로그래밍의 기초

The First Program

```
#include <stdio.h>

main()
{
    printf("Hello C++ World!!!...");
    return 0;
}
```

2) 프로그램의 구성요소

□ Simple C++ Program

```
/* 수의 제곱을 계산하는 프로그램 */  
#include <stdio.h>  
  
main()  
{  
    int a,b;  
  
    /* 값 입력 */  
    printf("Enter a number between 1 and 100: " );  
    scanf ("%d", &a);  
  
    /* 값을 계산하고 출력 */  
    b = a * a;  
    printf ("\n%d * %d = %d\n" , a, a, b);  
  
    return 0;  
}
```

2) 프로그램의 구성요소

□ **main()** 함수

- C++ 언어를 구성하는 모든 함수들의 우두머리 격이 바로 이 main 함수이다.
- main 함수는 C++ 프로그램의 가장 기본이 되는 함수로 반드시 있어야 하며 한 개만 존재해야 한다.
- Default Return-Type(ANSI 표준안): int

2) 프로그램의 구성요소

□ Statements

○ Statements

- C++ program 에서 프로그램의 가장 작은 실행단위
- Semicolon (;) 으로 끝난다.
- 보통 한 line이지만 여러line일수도 있다.

○ Compound Statements

- { } 으로 둘러싸여있다.

2) 프로그램의 구성요소

□ 주석문 (Comment)

- 프로그램에 대한 이해를 돕기 위해 덧붙이는 설명
- 프로그램의 동작 상태에 아무런 영향을 주지 않는다.
- C++은 두 가지의 주석문 양식을 제공

```
/* This is a multiline comment  
   slash Asterisk부터 asterisk slash 로 끝날 때까지 주석 */
```

```
//This is a single line comment
```

□ C++ keywords

- C++ 언어의 기본적인 동작을 수행하는데 사용되는 단어
- 예약어(reserved word)라고도 한다.
- Visual C++ Source code window 에서 Coloring 되어 나타난다.

2) 프로그램의 구성요소

□ #include

- <, > 사이의 파일의 내용을 포함시켜라
- Preprocessor에 의해 처리
- 이러한 파일을 Header File이라고 하며 주로 .h의 확장자를 가진다.

```
#include <iostream.h>
```

```
#include "mylib.h"
```

2) 프로그램의 구성요소

□ printf() 함수

- 화면상에 메시지나 결과를 출력하는 라이브러리 함수
- printf() 를 사용하기 위해서 stdio.h 파일을 포함(include)시켜야 한다.
- 확장 특수문자(escape sequence) 를 함께 출력할 수 있다.

```
#include <stdio.h>
int main()
{
    printf("C++");
    printf("C++\n");
    int a;
    a = 10;
    printf("%d", a);
    printf("a:%d", a);
    return 0;
}
```

2) 프로그램의 구성요소

□ 확장 특수 문자 (Escape Sequence)

문자	목적
\b	Back Space 문자
\n	Carriage Return & Line Feed 문자
\t	수평 tab 문자
\?	물음표 문자
\0	Null 문자
...	...

2) 프로그램의 구성요소

□ scanf()함수

- 키보드에서 데이터를 읽어 들이고 , 하나 이상의 프로그램 변수에 저장한다.
- scanf() 를 사용하기 위해서 stdio.h 파일을 포함(include)시켜야 한다.

```
#include <stdio.h>
int main()
{
    int a;
    printf ("Enter a number :");
    scanf ("%d:" , &a);
    printf("입력한 값은 %d", a);
    return 0;
}
```

2) 프로그램의 구성요소

□ return 문

- main() 함수의 return 문은 프로그램이 끝나기 전에 운영체제로 0 값을 되돌려 준다.

```
int main()
{
    ...
    return 0;
}
```

3) 변수와 상수

□ 변수

- 프로그램이 실행되는 동안 여러 가지 종류의 데이터를 저장하기 위해 사용
- 데이터가 저장되는 영역에 주어지는 이름
- 변수는 사용되기 전에 반드시 선언 되어야 한다.
- 변수 선언은 변수의 이름과 어떤 종류의 데이터를 저장할 것인지 함께 선언한다.
- 저장할 데이터의 형태를 고려해서 변수를 선언할 수 있도록 유형을 나누어 놓은 것이 자료형(Data Type) 이다.

3) 변수와 상수

□ 변수이름

- 문자, 숫자, 밑줄 을 포함할 수 있다.
- 첫번째 문자는 영문자나 밑줄이어야 한다.
- 대소문자를 구별한다.
- C++ keyword는 사용할 수 없다.
- 역할에 맞는 의미 있는 이름을 부여하는 것이 좋다

사용가능 변수명

의미

Percent

Annual_profit

_count

Count, count

첫번째 문자로 밑줄 가능
서로 다른 변수로 인식

사용불가능 변수명

의미

Saving#account

Value of sum

9winter

double

특수문자 사용할 수 없다
공백포함 할 수 없다.
첫 문자로 숫자가 올 수 없다
예약어는 사용할 수 없다

3) 변수와 상수

□ 변수유형

○ 기본적인 Data 유형

- int 와 같은 정수형
- double과 같은 실수형
- 문자를 위한 char 형

3) 변수와 상수

□ 기본적인 자료형 (Data Types)

char	1 byte	-128 ~ 127
unsigned char	1 byte	0 ~ 255
short	2 byte	-32,768 ~ 32,767
unsigned short	2 byte	0 ~ 65,535
int	4 byte	-2,147,483,648 ~ 2,147,483,647
unsigned int	4 byte	0 ~ 4,294,967,295
long	8 byte	$-2^{63} \sim 2^{63}-1$
unsigned long	8 byte	$0 \sim 2^{64}-1$
float	4 byte	
double	8 byte	

3) 변수와 상수

□ 변수 선언

- 변수는 사용하기 전에 선언해야 한다.(C vs. C++)
- 똑같은 유형의 여러 가지 변수는 함께 선언할 수 있다
- **Syntax**

```
Typename varname;
```

- **Ex)**

```
int i, j, k;  
char ch1, ch2;
```

3) 변수와 상수

□ typedef keyword

- 이미 존재하는 데이터 형에 새로운 이름을 지정하는데 사용

- Syntax

```
typedef datatype newtypename;
```

- Ex

```
typedef int integer;  
integer count;  
  
// int count;
```


3) 변수와 상수

□ 상수

- 변수와 마찬가지로 프로그램에서 사용되는 데이터를 저장하는 메모리 영역
- 변할 수 없는 고유한 값이다.
- 종류
 - 실제상수 (literal constant) :
필요할 때 마다 소스 코드에 직접 포함되는 실제 값
 - 기호상수(Symbolic constant):
독특한 이름이나 기호로 표현되는 상수
define이나 const 를 사용

3) 변수와 상수

□ 리터럴 (Literal)

Integer Literals

2000	310	-2396
020	030	
0xc8	0x0	
365L	0xA49c0l	
50000U	345u	

Floating-point literals

1.0	4.2
3.23E3	4.61e-9
6.23F	6.21f

Char literals

'a'	'\n'
-----	------

String literals

"Hello C++ world\n"

3) 변수와 상수

□ #define

- Preprocessor 에 의해 처리 된다.
- 단순한 code의 대체
- Syntax
 - #define** CONSTNAME literal
- Ex

```
#include <stdio.h>
#define PI 3.14
main()
{
    float w ;
    w = PI * 5 * 5;
    printf ("원의 넓이는 : %f" , w);
    return 0;
}
```

3) 변수와 상수

□ **const**

- 상수를 정의할 때 사용한다.
- 절대로 변경되어질 수 없다.
- Syntax
const data_type const_var = initial_value
- Ex)

```
#include <stdio.h>
main()
{
    const float PI=3.14159f;
    float r;
    r = 2 * PI * 5;
    printf ("원의 둘레는 : %f" , r);
    return 0;
}
```

4) 연산자

□ 연산자(Operator)

- 하나 이상의 연산항(Operand)에 대하여 어떤 동작이나 계산을 수행하도록 지시하는 기호
- 종류
 - 할당 연산자
 - 산술 연산자
 - 관계 연산자
 - 논리 연산자

4) 연산자

□ 할당 연산자 (Assignment Operators)

- 연산자로 = 를 사용한다.
- 초기화할 때 사용한다.
- 오른쪽의 표현식이 왼쪽의 변수에 할당된다.
- 오른쪽의 표현식은 항상 할당되기 전에 먼저 계산된다.
- 함께 할당할 수도 있다.

Ex)

```
int x;  
int y = 25;  
int z ( 26 );  
x = 24;  
x = x + 10;  
int l, j, k;  
i = j = k = 0;
```

4) 연산자

□ 산술연산자

- 정수형, 실수형을 피연산자로 가진다.
- %는 정수형을 피연산자로 가진다.
- 이항 산술 연산자
 - + Addition
 - - Subtraction
 - * Multiplication
 - / Division
 - % Modulus
- 단항 산술 연산자
 - ++
 - --

4) 연산자

□ 단항 산술 연산자

- ++ 연산자는 값을 1 증가시킨다.

```
int i = 3;  
i++;           // i = 4
```

- ++ 은 두가지 방법으로 사용될 수 있다.

```
int i = 3, j = 0;  
j = ++i;       // i를 증가시킨 후 j에 할당한다.  
               // i = 4, j = 4  
j = i++;       // j에 할당한후 i를 증가시킨다.  
               // i = 5, j = 4
```

- -- 연산자도 같은 방법으로 사용

4) 연산자

□ 비교연산자

- 피연산자로 정수형, 실수형을 가진다.
- 결과값은 true, false값을 가지며, true는 1, false는 0으로 간주된다.

- Equal to ==
- Not Equal to !=
- Less than <
- Greater than >
- Less than or Equal to <=
- Greater than or Equal to >=

```
int i = 3, j = 10;  
int ret;  
ret = ( i == j );           // ret = false  
ret = ( i < j );            // ret = true  
if ( i != j ).....
```

4) 연산자

□ 논리 연산자

- 피연산자는 논리값 또는 관계식이다.
- 0은 false로, 0을 제외한 모든 값은 true로 처리

- Logical AND &&
- Logical OR ||
- Logical NOT !

```
int x=0;
if ( (x >= 0) && (x <= 100))
    // statement and expression
if ( (x % 2 ==0) || (x % 5 == 0))
    // statement and expression
If ( ! (x % 2) )
    // statement and expression
```

4) 연산자

□ 복합 할당 연산자.

- += Add and Assign
- -= Subtract and assign
- *= Multiply and assign
- /= Divide and assign
- %= Modulus and assign

Ex)

```
int total=10, num=1;  
.....  
total = total + num;  
total += num
```

4) 연산자

□ 삼항 연산자 (? :)

- If ... else 구문과 유사
- syntax

Expression ? A : B

조건이 true 이면 결과는 A false 이면 결과는 B

```
cout << "Maximum value is " << ( x >= ? x : y );
```

4) 연산자

□ 연산자 우선순위

- 산술연산자 `!, ++, --, +, -`
- 산술연산자 `*, /, %`
- 비교연산자 `<, <=, >, >=`
`==, !=`
- 논리연산자 `&&`
`||`

5) 제어문

□ 제어문

- 프로그램은 기본적으로 입력된 순서대로 실행된다.
- 실제 프로그램에서는 특정문장들을 반복해서 실행해야 할 경우가 많이 발생한다.
- 제어문은 프로그램의 실행순서를 제어해 준다.
- 종류(Block, Loop)
 - if..... else
 - switch
 - for
 - while
 - do while
 - break
 - continue

5) 제어문

□ if else Statement

○ Syntax

```
if ( expression )  
    statement;      // Action1  
else  
    statement;      // Action2
```

```
if ( x >= y )  
    max =x;  
else  
    max =y;
```

5) 제어문

□ 중첩 if statement

```
if ( speed > 25 )  
    if ( speed > 65 )  
        cout << "Speed is over 65 \n";  
    else  
        cout << "Speed is over 25 \n";  
else  
    cout << "Speed is under 25 \n";
```

```
if ( speed > 65 )  
    cout << "Speed is over 65 \n";  
Else if ( speed > 25 )  
    cout << "Speed is over 25 \n";  
else  
    cout << "Speed is under 25 \n";
```


5) 제어문

□ Switch statement

- 여러 정수 값 중에서 선택하는데 유용하다.
- Syntax

```
switch ( int_expr ) {  
    case constant_expr1:  
        statement1;  
        break;  
    case constant_expr:  
        statement2;  
        break;  
    default:  
        statement3;  
        break;  
}
```

5) 제어문

□ While loop

○ Syntax

While (expression)
statement;

```
int i =0;  
while ( i < 10 ){  
    cout << "i = " << i << endl;  
    i++;  
}
```

5) 제어문

❑ do.... while loop

○ Syntax

```
Do {  
    statement;  
} while ( expression );
```

```
int i =0;  
do{  
    cout << "i = " << i << endl;  
    i++;  
} while ( i < 10 );
```

5) 제어문

□ for Loop

○ Syntax

```
for ( init_expr ; boolean_expr ; update_expr )  
    statement;
```

```
int i =0;  
for ( i = 0 ; i < 10 ; i ++ )  
    cout << "i = " << i << endl;
```

5) 제어문

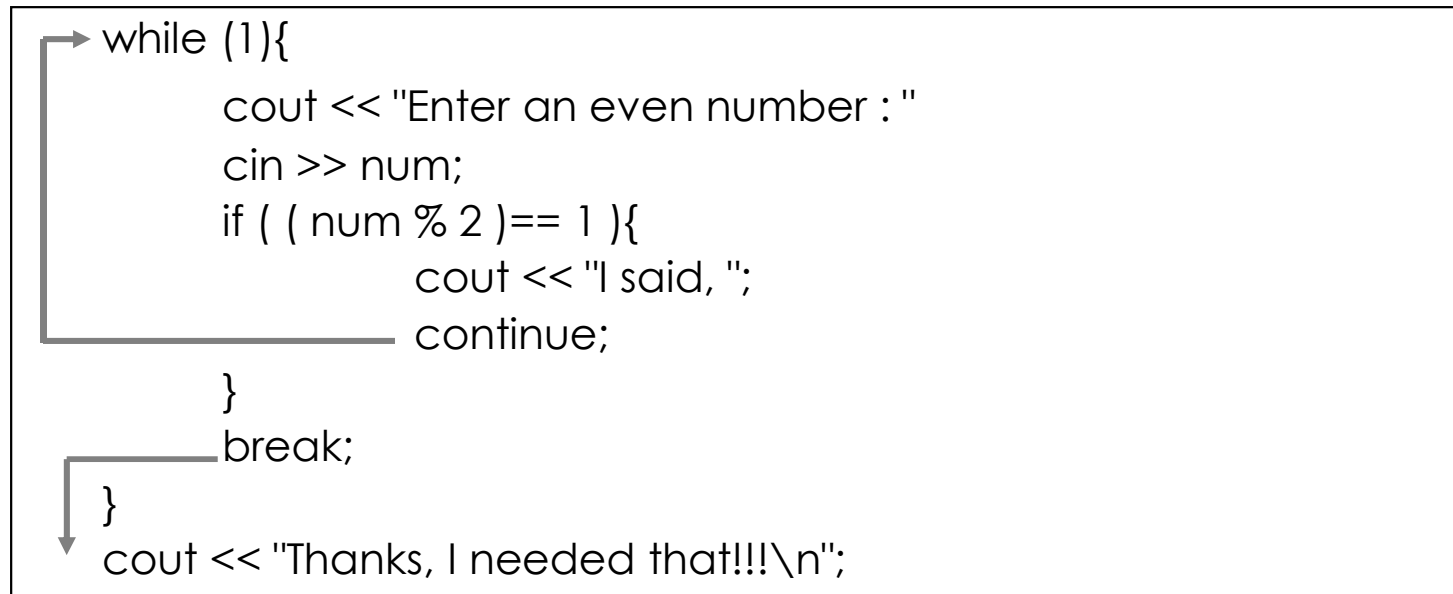
□ break and ... continue Statements

○ ...break

- Loop나 switch 문장을 벗어난다.
- 컨트롤을 loop body나 switch 문장 다음의 첫번째 문장으로 전달한다.
- 어떤 상황에서는 코드를 단순하게 한다.

○ ...continue

- Loop에서만 사용되어진다.
- 현재의 loop 반복을 포기하고 다음 loop 반복으로 이동한다.



6) 함수

□ 함수

- 특정한 기능을 가진 독립적인 모듈
- 프로그램이 커지고 복잡해짐에 따라 반복되는 내용을 작고 의미 있는 단위로 나누어서 독립된 단위로 구성
- main 도 일종의 함수이며,
- main 함수를 제외한 모든 함수는 호출에 의해 수행된다.
- 표준라이브러리 함수, 사용자 정의함수가 있다

6) 함수

□ 함수의 정의

- 함수는 이름을 가진다.
- 함수는 독립적이다.
- 특정 동작을 수행한다.
- 호출한 프로그램으로 결과값을 되돌린다.
- Syntax

반환할 데이터형 **funcName** (*인자리스트*)

```
{  
    statement1;  
    statement2;  
    ...  
    return 반환할 값;  
}
```

6) 함수

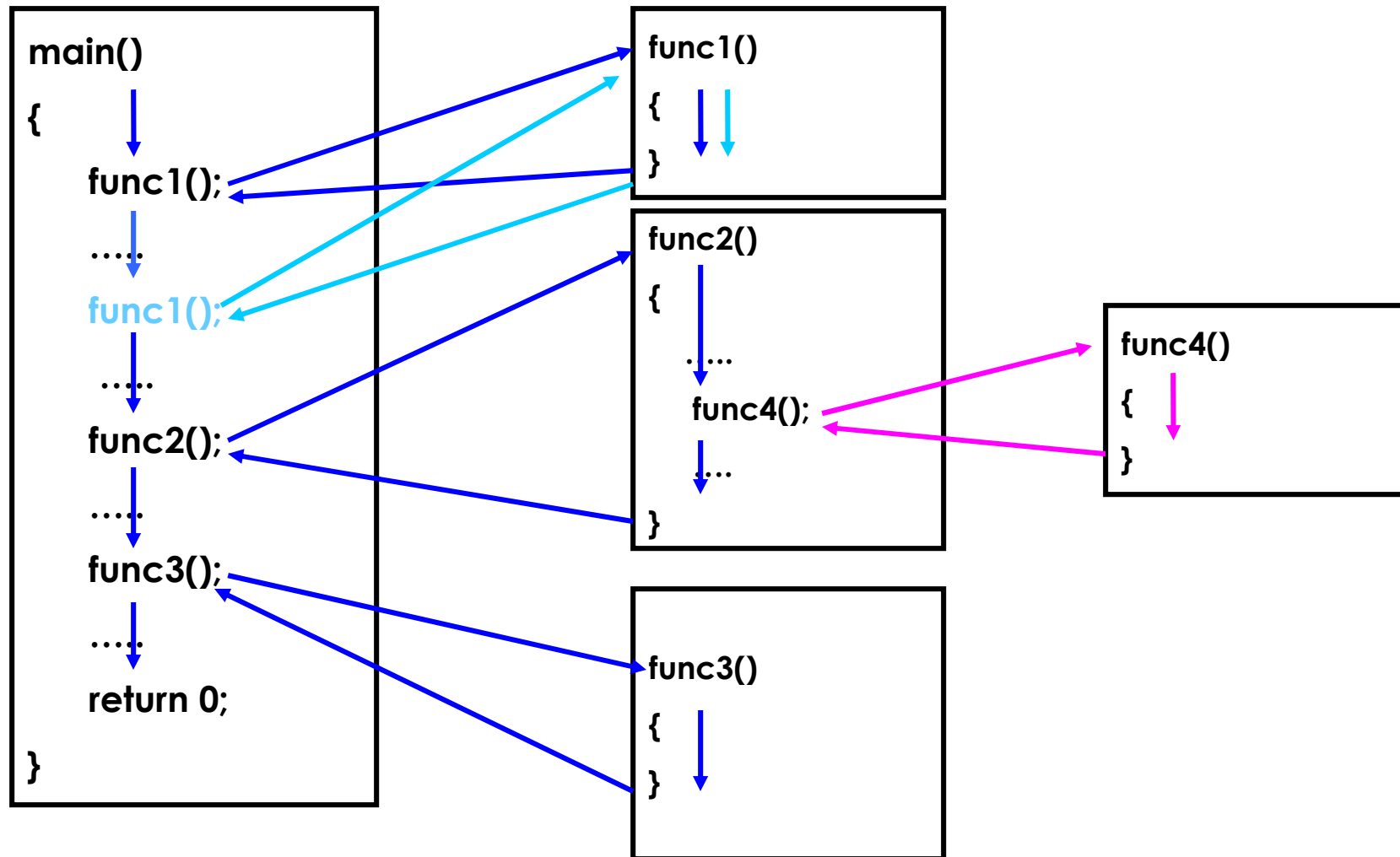
□ 함수의 용법

- 원형 선언 : 호출이 올바르게 되고 있는지 검사하기 위한 것
함수명과 괄호를 가지며,
하나 이상의 입력 인자와 하나의 리턴값을 가질 수 있다.
- 몸체 정의 : 함수가 수행할 내용을 기술 그 내용은 {} 안에 나타냄
- 호출 : 함수를 사용하는 것으로서 입력 인자를 넘겨주고 반환값을 받는다.

Ex)

```
#include <stdio.h>
void showname(); // 함수 원형선언
int main (){
    showname (); // 함수 호출
}
void showname (){ // 함수 몸체 정의
    printf("Hello My name is C++\n");
    return ;
}
```


6) 함수



6) 함수

□ 함수에 인수 전달하기

- 호출하는 함수에서 정의하는 함수에 어떠한 값을 넘겨주고자 할 때 사용
- 여러 개의 인수를 사용할 경우 인수는 순서대로 매개변수에 할당

함수호출

Func1(3, 5, 10)

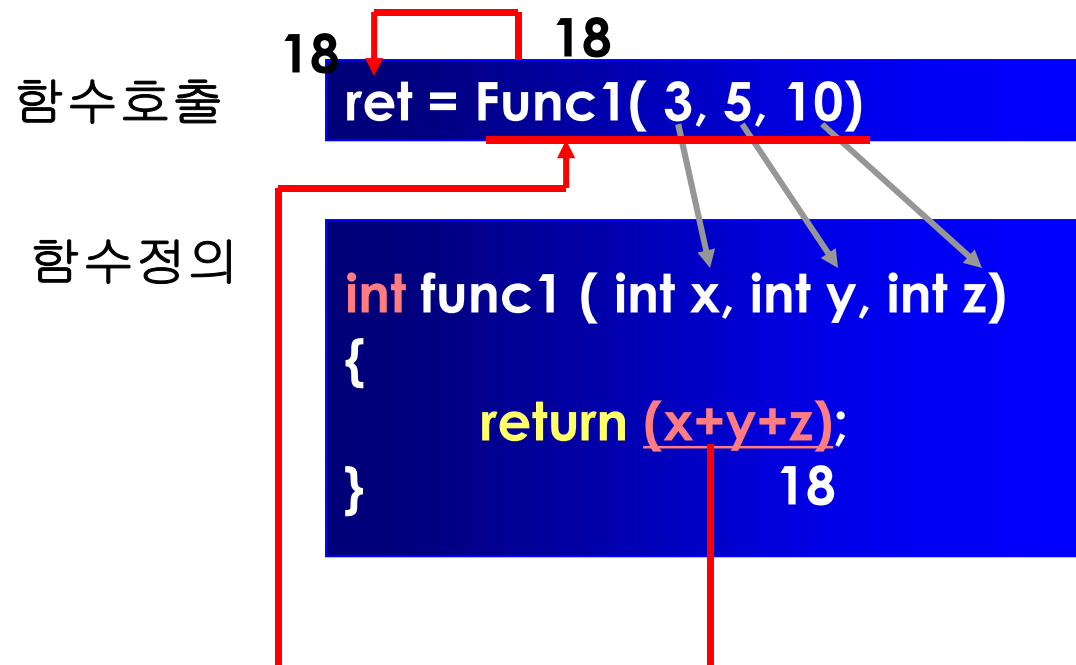
함수정의

void Func1 (int x, int y, int z)

6) 함수

□ 함수에서 값의 전달

- 결과값을 프로그램으로 돌려주기 위해서는 return문과 함께 C의수식을 사용한다.
- 함수의 return값은 수식의 결과이다.



6) 함수

```
#include <stdio.h>

int square(int n);    // 함수 원형선언

main ()
{
    int ret;
    ret = square ( 5 );    // 함수 호출
    printf ("결과값은 : %d", ret ) ;
    return 0;
}

int square ( int n)    // 함수 몸체 정의
{
    return n*n;
}
```

6) 함수

□ Simple C++ Program structure

Source Code

```
#include <iostream.h>
```

```
#define SIZE 100
```

```
함수 원형 선언
```

```
전역변수 선언
```

```
int main()  
{  
    . . .  
    return 0;  
}
```

```
함수 몸체 정의
```

7) 입출력의 기초

□ printf() 함수

- 화면상에 메시지나 결과를 출력하는 라이브러리 함수
- printf() 를 사용하기 위해서 stdio.h 파일을 포함(include)시켜야 한다.
- 일반적인 텍스트는 형식화 문자열에 입력되어 그대로 출력된다.
- 확장 특수문자(escape sequence) 를 함께 출력할수 있다.
- 변환 문자 (Conversion Specifier)는 %와 하나의 문자로 구성된다.

7) 입출력의 기초

□ 확장 특수 문자 (Escape Sequence)

문자	목적
\a	경고음(Bell)
\b	Back Space 문자
\n	Carriage Return & Lien Feed 문자
\t	수평 tab 문자
\\	백슬래시
\?	물음표 문자
\'	작은따옴표
\"	큰 따옴표
\0	Null 문자

7) 입출력의 기초

□ 변환문자 (Conversion Specifier)

문자	목적	목적
%c	하나의 문자	char
%d	부호 있는 십진 정수형	int, short
%ld	부호 있는 long 십진 정수형	long
%f	부동 소수형 십진수	float, double
%s	문자열	char 배열
%u	부호 없는 십진 정수형	unsigned int, unsigned short
%lu	부호 없는 long 십진 정수형	unsigned long

- 왼쪽에서 오른쪽으로 변수와 쌍을 이루게 된다.

```
printf ("Rate = %f , amount = %d ", rate, amount );
```


7) 입출력의 기초

□ puts() 함수

- 화면상에 텍스트 메시지를 출력하는데 만 사용
- 마지막에 new line을 추가하여 문자열을 출력한다.

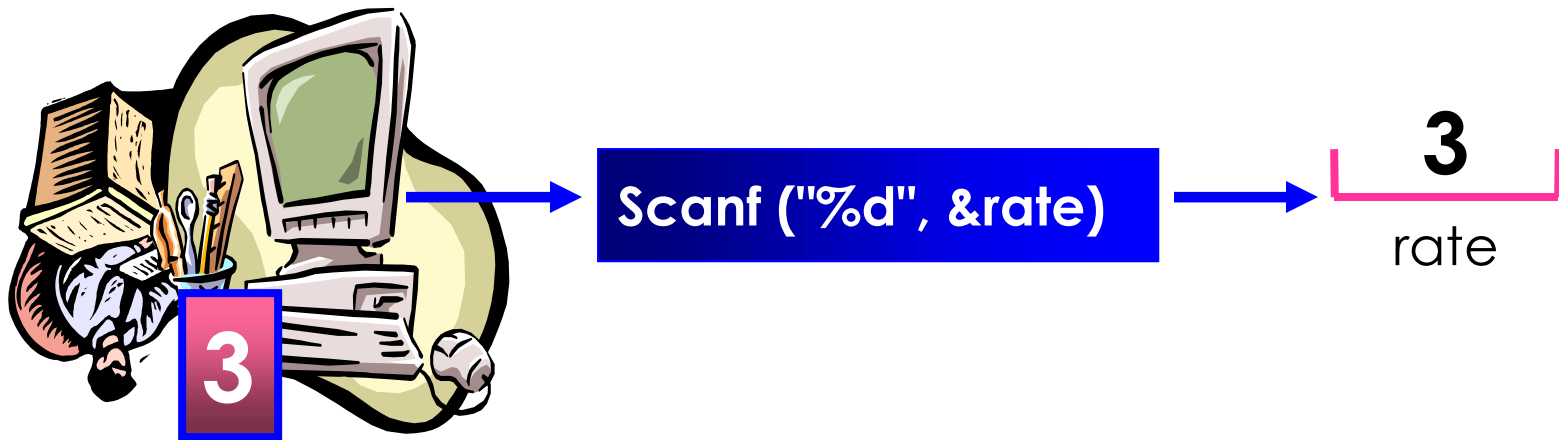
```
#include <stdio.h>

main()
{
    puts ("Hello, world..");
    printf ("Hello, world..\n");
    // 위의 두 함수는 같은 역할을 한다.
    return 0;
}
```

7) 입출력의 기초

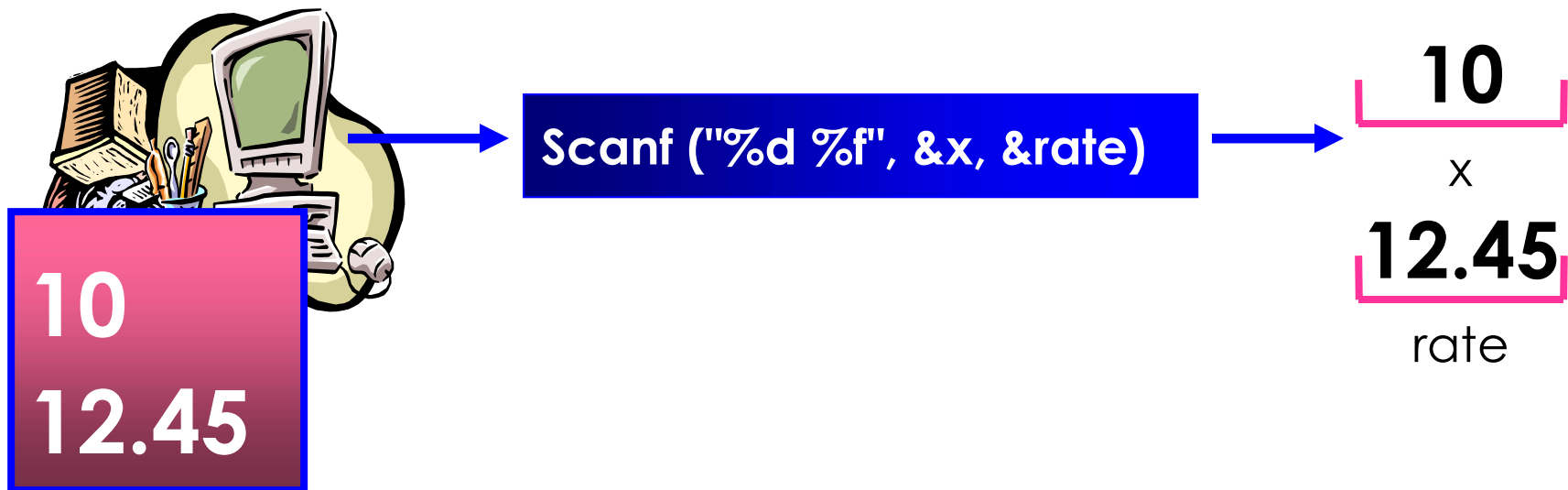
□ scanf()함수

- 키보드에서 데이터를 읽어 들이고 , 하나 이상의 프로그램 변수에 저장한다.
- scanf() 를 사용하기 위해서 stdio.h 파일을 포함(include)시켜야 한다.
- 입력형식을 지정하는 형식화 문자열을 사용한다.
- 변수 이름 앞에 & 기호를 붙여야 한다.
- 여러 개의 값이 입력될 때 입력되는 값을 구분하기 위해 공백을 이용한다.
(공백은 빈칸(space), 탭(tab), 문자진행(newline)이 될 수 있다.)



7) 입출력의 기초

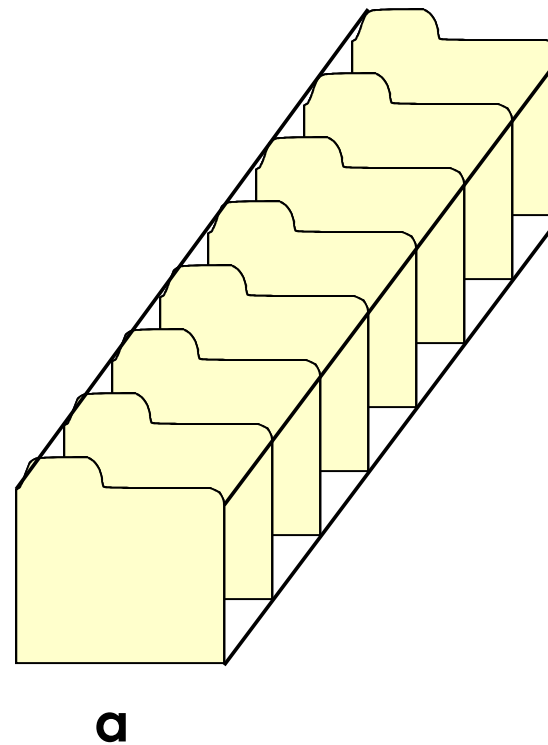
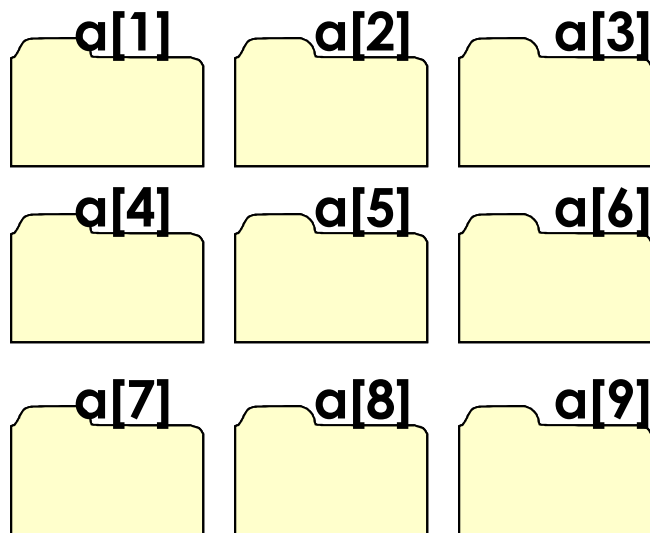
- 여러 개의 변수를 사용하는 경우



8) 배열

□ 배열 (Array)

- 배열이란 : 같은 자료형을 가진 Data 들의 연속된 집합
- 각 저장영역을 배열의 요소라고 한다.



8) 배열

□ 배열의 선언 및 사용

○ Syntax

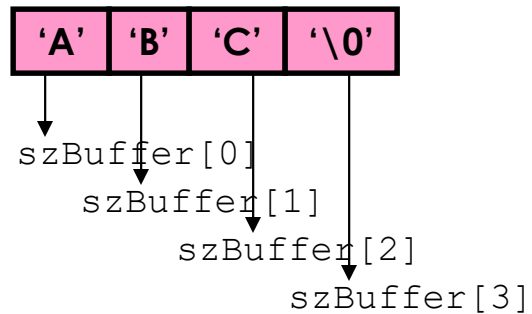
Datatype array_name[배열의요소의갯수];

○ 배열의 요소는 0부터 시작한다.

```
int nSales[4];
```

10	20	30	40
nSales[0]	nSales[1]	nSales[2]	nSales[3]

```
char szBuffer[4];
```



8) 배열

```
#include <stdio.h>

main()
{
    int M[5]; //배열 선언, 배열명 M 크기 5 -> 곧 5개의 기억공간을 가진다
    printf("Input Number:");
    //동일한 작업이 반복되므로 for문을 사용한다
    //주의사항 : 배열은 0번 번지부터 시작한다
    for (int i = 0; i < 5 ; i++){
        printf("M[%d] = ", i);
        scanf("%d", &M[i]);           //키보드로 입력 받아 배열에 저장
    }
    printf("입력된 값을 출력하여 보자.\n");
    for (i = 0; i < 5;i++)
        printf("M[%d] = %d\n" , i, M[i]);    //배열의 값 출력
    return 0;
}
```

8) 배열

□ 배열의 초기화

- 배열은 선언할 때 전체나 일부분을 초기화 할 수 있다.
- 초기값은 등호와 함께 중괄호 내에 포함되며, 쉼표로 구분된다.

```
int nSales[4]={10,20,30,40};
```

10	20	30	40
nSales[0]	nSales[1]	nSales[2]	nSales[3]

```
int nSales[]={10,20,30};
```

10	20	30
nSales[0]	nSales[1]	nSales[2]

```
int nSales[4]={10,20};
```

10	20	0	0
nSales[0]	nSales[1]	nSales[2]	nSales[3]

```
int nSales[4]={10,20,30,40,50,60}; //error
```

8) 배열

□ 2차원 배열

- 두개의 첨자를 가진다.
- 메모리에 순서대로 저장된다.
- Syntax

Datatype array_name [행의갯수][열의갯수];

```
int a[3][4];  
a[1][2] = 100;
```

	0열	1열	2열	3열
0행	a[0][0]	a[0][1]	a[0][2]	a[0][3]
1행	a[1][0]	a[1][1]	a[1][2] 100	a[1][3]
2행	a[2][0]	a[2][1]	a[2][2]	a[2][3]

8) 배열

□ 2차원 배열의 초기화

- 뒷부분의 첨자가 먼저 증가하는 순서로 배열의 요소에 할당된다.

```
int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

	0열	1열	2열	3열
0행	a[0][0] 1	a[0][1] 2	a[0][2] 3	a[0][3] 4
1행	a[1][0] 5	a[1][1] 6	a[1][2] 7	a[1][3] 8
2행	a[2][0] 9	a[2][1] 10	a[2][2] 11	a[2][3] 12

```
int a[3][4]={{1,2,3,4},  
             {5,6,7,8},  
             {9,10,11,12}};
```

8) 배열

```
#include <stdio.h>
main()
{
    int a[3][4] = {10,20,30,40,2,4,6,8,1,3,5,7};
    //2차원 배열의 초기 값 주는 형태1
    int b[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
    //2차원 배열의 초기 값 주는 형태2
    int c[3][4]; //합한 결과를 저장할 2차원배열 선언

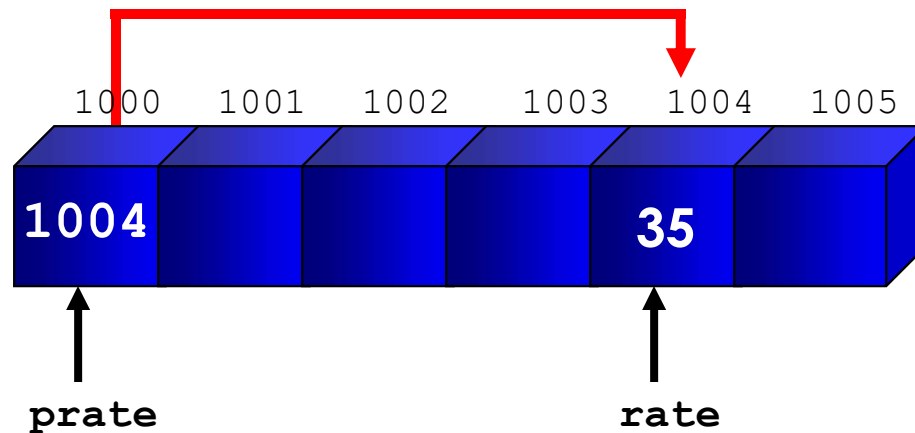
    //이차원 배열의 합
    for(int row = 0; row < 3; row++)
        for(int col = 0; col < 4; col++)
            c[row][col] = a[row][col] + b[row][col];

    for(row = 0; row < 3; row++)
        for(int col = 0; col < 4 ; col++)
            printf ("%d, ", c[row][col]);
        printf("\n");
}
```

9) 포인터

□ 포인터

- 변수를 선언할 때 컴파일러는 변수를 저장하기 위해 특정 주소의 메모리 영역을 보존한다.
- 컴파일러는 변수의 이름을 특정 주소와 관련시킨다.
- 그런데 C에서는 이 주소값 역시 어떤 변수에 저장할 수 있다.
- 즉 포인터란 다른 어떤 변수의 주소값을 가지는 변수이다.



9) 포인터

□ 포인터 변수

- 포인터 변수에 저장되는 내용도 정수형태의 값이 저장된다.
- 그 크기는 정해져 있다.
- Syntax

`datatype *pointervarname;`

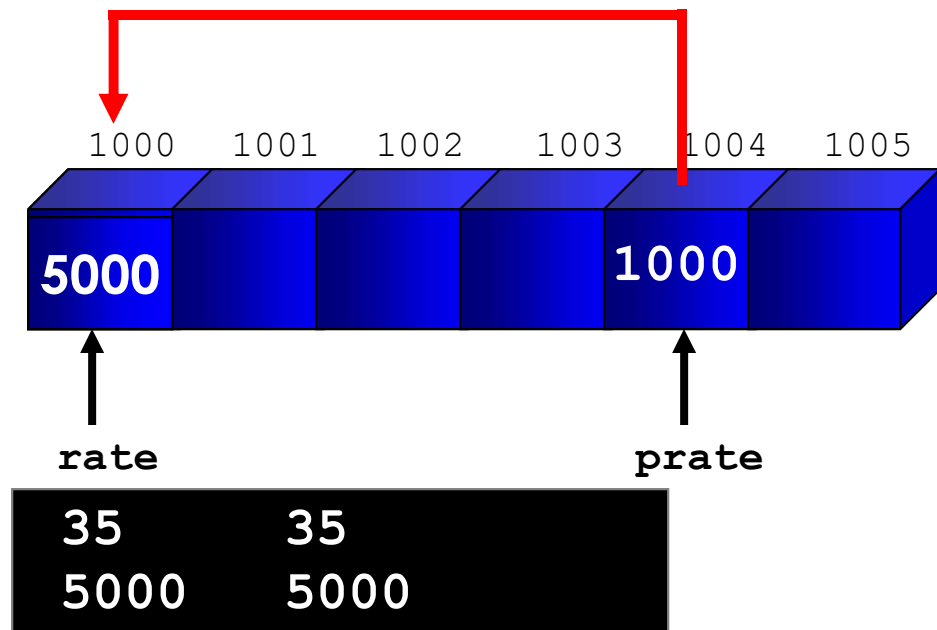
```
int *prate;  
char *pname;  
float *pvalue;
```

9) 포인터

□ 포인터의 초기화와 사용

- 초기화할 때는 &(주소 연산자) 를 사용한다.
(&는 변수 이름 앞에 사용되며, 그 변수의 주소값을 돌려준다.)
- 사용할 때는 *(간접연산자)를 사용한다.
(*를 포인터 변수 앞에 사용하면 포인터가 가리키는 변수의 값을 의미한다.)

```
int rate=35;  
int *prate;  
prate = &rate;  
printf("%d\t", rate);  
printf("%d\n", *prate);  
rate= 2;  
*prate=5000;  
printf("%d\t", rate);  
printf("%d\n", *prate);
```



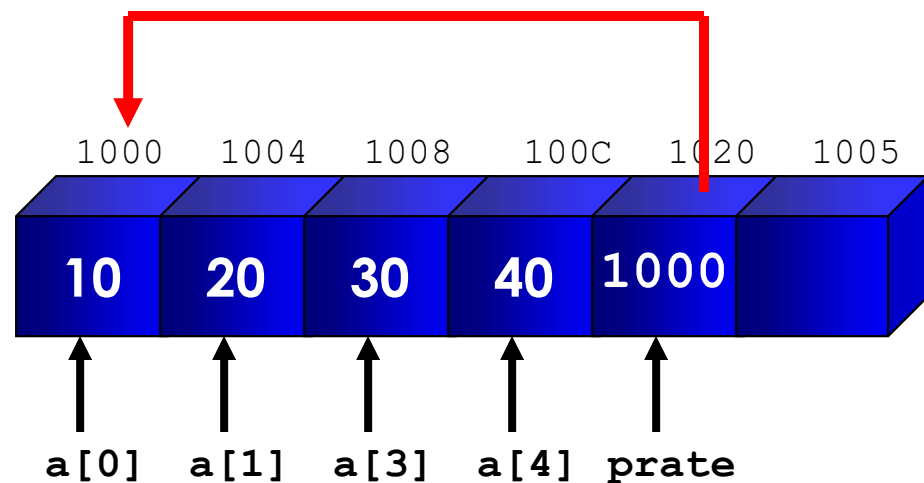
9) 포인터

□ 포인터와 배열

- 대괄호를 포함하지 않은 배열의 이름은 배열의 첫번째 요소에 대한 포인터이다
배열의 이름은 포인터 상수이며
프로그램이 실행되는 동안 배열의 주소는 변경될 수 없다.
- 배열의 요소가 메모리에 저장되는 방법
배열은 연속된 기억공간에 저장된다.

```
int a[4]={10,20,30,40};  
int i;  
int *p;  
p = a;  
printf ("%d\n", a);  
for(i=0;i<4;i++)  
    printf ("%d\t", &a[i]);
```

```
1000  
1000 1004 1008 100C
```



9) 포인터

□ 포인터의 연산

○ 포인터의 증가

- 포인터의 값을 1 증가시키면 배열의 다음 요소를 지적
- 포인터가 가리키는 데이터형의 크기에 따라서 저장된 주소값을 증가

○ 포인터의 감소

- 포인터의 값을 1 감소시키면 배열의 이전 요소를 지적
- 포인터가 가리키는 데이터형의 크기에 따라서 저장된 주소값을 감소

```
int a[3]={10,20,30};  
int *p;  
int i;  
p = a;  
for(i=0 ;i<3 ; i++)  
    printf ( "%x : %d \n" , p,*p++);
```

9) 포인터

□ 배열의 첨자표기방법과 포인터

- 괄호를 포함하지 않는 배열의 이름은 첫번째 요소에 대한 포인터이다.
그래서 간접 연산자(*)를 사용하면 배열의 첫번째 요소를 사용할 수 있다.
- `*(array + i) == array[i]`

```
int a[3]={10,20,30};  
int i;  
for(i=0 ;i<3 ;i++)  
    printf ( "%d , %d \n" , a[i], *(a+i));
```


9) 포인터

□ 함수에 배열을 전달하는 방법

- 함수에 배열을 전달하려면 포인터를 사용해야 한다.
(call by address 를 사용)

```
#include<stdio.h>
void prn(int *);
void main()
{
    int a[5] = {10,20,30,40,50};
    prn(a); //call by address
}
void prn(int *p)
{
    int i;
    for( i = 0; i < 5; i++)
        printf ("%d\n",*(p+i));
}
```

10) 문자와 문자열

□ 문자

- 문자는 각각에 대응하는 숫자코드(ASCII CODE)가 존재한다.
대,소문자, 숫자 구두점, 그 밖의 기호에는 0~255 사이의 ASCII CODE값이 할당되어 있다.
- 문자값을 사용하기 위해서는 변수를 char 형 변수로 선언한다.
- 초기화하기 위해서는 하나의 문자를 작은 따옴표 내에 포함시킨다.
- 출력할 때는 printf() 함수에 형식화 문자열인 %c를 이용한다.

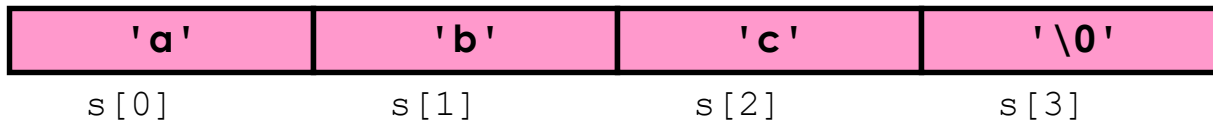
```
#include<stdio.h>
void main()
{
    char ch1 = 'a';
    char ch2 = 90;
    printf ("ch1 : %c, %d\n", ch1, ch1);
    printf ("ch2 : %c, %d\n", ch2, ch2);
}
```

10) 문자와 문자열

□ 문자열

- char 형 변수는 단지 하나의 문자값만을 저장한다.
- 문자들의 연결된 집합인 문자열을 저장하기 위해서는 문자형 배열을 이용한다.
- 문자열을 저장하기 위해서는 일련의 문자열 외에 '\0' (NULL 문자)가 추가된다.
(예를 들어 'a', 'b', 'c' 를 저장하기 위해서는
마지막에 NULL 문자를 저장하기 위해 4개의 요소가 필요하다)
- 문자배열의 초기화

```
char s[4]={'a', 'b', 'c', '\0'};
```



```
char s[4]="abc";
```

```
char s[]="abc";
```

10) 문자와 문자열

□ 문자열의 출력

○ puts() 함수

- 인수는 문자열에 대한 포인터를 받아들인다.
- NULL을 만날 때까지 출력한다.
- 마지막에 new line 을 포함시킨다.

○ printf() 함수

- 인수는 문자열에 대한 포인터를 받아들인다.
- NULL을 만날 때까지 출력한다.
- 문자열을 출력하기 위해서는 형식화문자 %s를 이용한다.

```
char s[4] = "abc";  
puts(s);  
printf("%s", s);
```

10) 문자와 문자열

□ 문자열의 입력

○ 문자열을 읽어 들이기 위해서는 우선 문자열을 저장하기위한 영역을 확보해야 함

○ gets() 함수

- 문자열을 저장할 배열의 첫번째 요소에 대한 포인터를 인수로 받아들인다
- 키보드에서 newline 을 입력할 때까지의 문자열을 읽어 들인다.
- 마지막 newline을 제거하고 NULL 문자를 추가시킨다.
- 읽어 들인 문자열에 대한 포인터를 반환한다.

○ scanf() 함수

- 문자열을 저장할 배열의 첫번째 요소에 대한 포인터를 인수로 받아들인다
- %s를 이용한다.
- 공백이 아닌 첫번째 문자에서부터 시작된다.
- 공백(newline, space, tab)이 입력되면 끝나고 공백을 제거하고 NULL을 추가시킨다.
- 성공적으로 입력된 데이터의 수를 반환한다.

10) 문자와 문자열

```
#include <stdio.h>
int main()
{
    char buff[100];
    puts("Enter a string:");
    gets(buff);
    printf ("You entered follow string : %s", buff);
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    char fn[100], ln[100];
    puts("Enter a first name & last name:");
    scanf("%s%s", fn, ln);
    printf ("You entered first name: %s last name: %s", fn ,ln );
    return 0;
}
```

10) 문자와 문자열

□ 문자열의 조작

- String.h를 포함시켜야 한다.
- strlen()함수
 - 문자열의 길이를 구해준다.
 - int strlen(char *str);
인자는 길이를 계산할 문자열에 대한 포인터, 반환값은 문자열의 길이
- strcpy () 함수
 - 문자열을 복사해 준다.
 - char *strcpy (char *Destination, char *Source)
Source가 가리키는 문자열을 Destination 이 가리키는 곳으로 NULL을 만날 때 까지 복사한다. 반환값은 Destination에 대한 포인터
- strcat()함수
 - 문자열을 결합해 준다.
 - char *strcpy (char *Destination, char *Source)
Source가 가리키는 문자열을 Destination 이 가리키는 문자열의 마지막에 추가한다.
반환값은 Destination에 대한 포인터
결과적으로 생성되는 문자열을 저장하기에 충분한 메모리 영역이 할당되어야 한다.
- strcmp()함수
 - 두 문자열을 크기를 비교한다.(ASCII Code값 사용)
 - int strcmp (char *str1, char *str2)
반환값은 str1 > str2 → 0보다 큰값 , str1 == str2 → 0, str1 < str2 → 0보다 작은값

10) 문자와 문자열

```
#include <stdio.h>
#include <string.h>
main()
{
    char buff[100];
    puts("Enter a Name:");
    gets(buff);
    printf ("You entered follow Name : %s\n", buff);
    printf ("Name length is : %d \n", strlen(buff));
    strcpy (buff, "Small Business Corporation");
    printf ("buff: %s\n", buff);
    strcat (buff, " ++");
    printf ("buff: %s\n", buff);
}
```


10) 문자와 문자열

```
#include <string.h>
#include <stdio.h>
char string1[] = "The quick brown dog jumps over the lazy fox";
char string2[] = "The QUICK brown dog jumps over the lazy fox";
void main( void )
{
    char tmp[20];
    int result;
    /* Case sensitive */
    printf( "Compare strings:\n\t%s\n\t%s\n\n", string1, string2 );
    result = strcmp( string1, string2 );
    if( result > 0 )
        strcpy( tmp, "greater than" );
    else if( result < 0 )
        strcpy( tmp, "less than" );
    else
        strcpy( tmp, "equal to" );
    printf( "\tstrcmp: String 1 is %s string 2\n", tmp );
    /* Case insensitive (could use equivalent _stricmp) */
    result = _stricmp( string1, string2 );
    if( result > 0 )
        strcpy( tmp, "greater than" );
    else if( result < 0 )
        strcpy( tmp, "less than" );
    else
        strcpy( tmp, "equal to" );
    printf( "\t_stricmp: String 1 is %s string 2\n", tmp );
}
```

11) 구조체

□ 구조체 (structures)

- 프로그램에서 필요한 데이터형을 프로그래머가 직접 구성하여 사용하는 데이터의 저장 방식
- 여러 개의 변수를 쉽게 사용할 수 있도록 하나의 이름으로 묶은 하나 이상의 변수의 집합
- 구조체에 포함되는 각각의 변수를 구조체의 Member라고 한다.
- 구조체를 사용하려면
 - 구조체 선언
 - 구조체 변수 선언
 - 구조체의 각 멤버를 사용

11) 구조체

□ 구조체 선언

```
struct structName {  
    Datatype membervarname1;  
    Datatype membervarname2;  
    ....  
};
```

□ 구조체 변수 선언

```
struct structName structvarName;
```

□ 구조체 멤버의 사용

- 일반적인 변수처럼 사용될 수 있다.
- 구조체 멤버 연산자 (.)dot operator를 이용해 사용한다.

```
struct structName structvarName;  
structvarName.mebercarname=10;
```

11) 구조체

```
#include <stdio.h>

struct coord {
    int x;
    int y;
};

main()
{
    struct coord c1;
    c1.x= 10;
    c1.y = 20;
    printf ("x coord is %d,  y coord is %d\n", c1.x, c1.y);
    return 0;
}
```

11) 구조체

□ 구조체의 초기화

- 구조체 역시 선언과 동시에 초기화 할 수 있다.
- 등호와 함께 중괄호 안에 초기화 값의 목록이 나타난다. 각 값은 쉼표에 의해서 구분된다.

```
struct coord {  
    int x;  
    int y;  
};  
struct person {  
    char name[80];  
    int age;  
};  
struct coord c1 = {3,4};  
struct person p1 = {"hongkildong", 35};
```

11) 구조체

□ 구조체의 사용

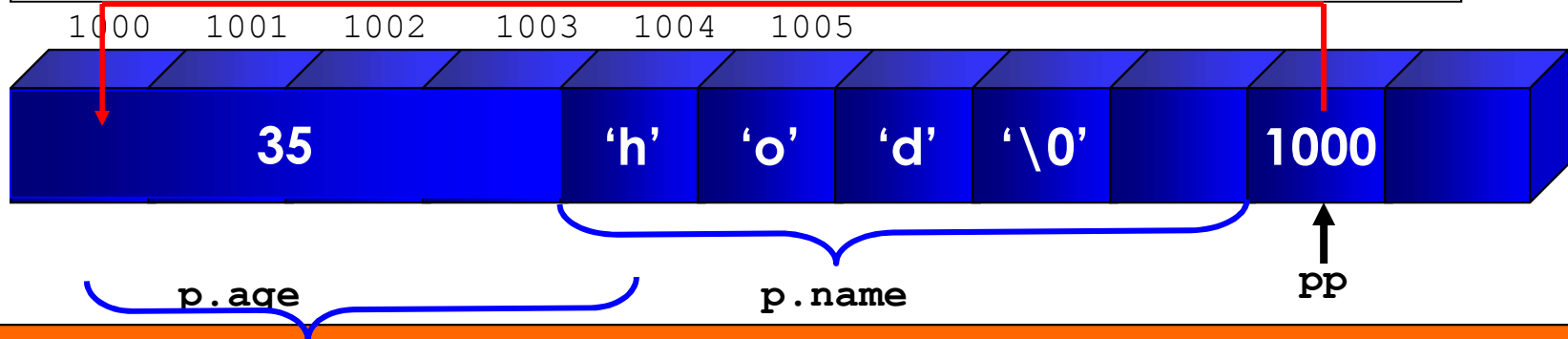
- 같은 유형의 구조체 변수끼리 값 대입
- 구조체를 가지는 구조체
- 배열을 가지는 구조체

11) 구조체

□ 구조체에 대한 포인터

- 다른 데이터형에 대한 포인터와 마찬가지로 구조체에 대한 포인터를 선언하고 사용할 수 있다.
- 이 경우 멤버에 접근하기 위해서는 간접 멤버 참조 연산자(->)를 사용한다.

```
struct person {  
    int age;  
    char name[5];  
};  
struct person p = {35, "hod"};  
struct person *pp = &p;  
printf ("name:%s, age: %d\n", p.name, p.age);  
printf ("name:%s, age: %d\n", (*pp).name, (*pp).age );  
printf ("name:%s, age: %d\n", pp->name, pp->age);
```

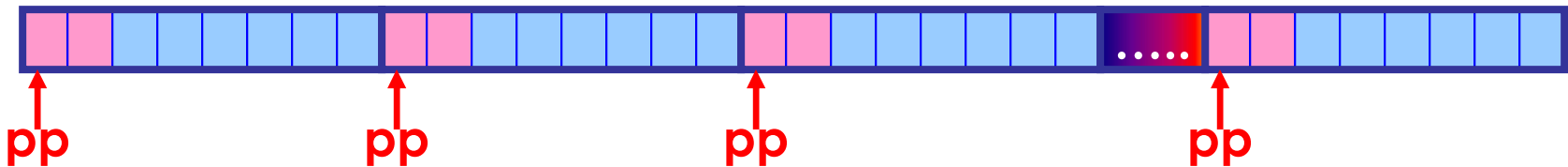


11) 구조체

□ 구조체 배열

- 구조체 역시 배열로 선언해 사용할 수 있다.
- 구조체 배열 역시 일반 변수와 마찬가지로 구조체 배열명은 그 배열의 시작 주소값과 동일하다.
- 주소값에 +1을 하였을 경우 구조체 크기만큼의 바이트수가 증가한다.

```
struct person {  
    char name[5]; int age; };  
int i;  
struct person p[2] = {"abc", 1, "def", 2};  
struct person *pp= p; //struct person *pp= &p[0];  
for ( i = 0 ; i < 2; i ++)  
{  
    printf ("name:%s ,age: %d \n", p[i].name, p[i].age );  
    printf ("name:%s ,age: %d \n", pp->name, pp->age );  
    pp++;  
}
```



11) 구조체

□ 함수의 인수로 구조체 전달하기

- 다른 데이터형과 마찬가지로 함수의 인수로 구조체를 전달할 수 있다.

```
#include <stdio.h>
struct person {
    int age;
    char name[5];
};

void prn ( struct person );
void main()
{
    struct person p1 ={35, "hgd"};
    prn (p1);
}
void prn ( struct person p)
{
    printf ( "name:%s ,age: %d\n", p.name, p.age );
}
```

12) 변수의 범위

□ 변수의 범위

- 변수의 범위란 프로그램에서 변수를 사용할 수 있는 범위나 또는 변수가 프로그램 내에서 효과를 낼 수 있는 범위
- 변수의 범위는 변수의 생명 즉 메모리에서 변수의 값이 보존되는 기간이나 변수에 저장 영역이 할당되고 해제 되는 시기에 영향을 준다.

12) 변수의 범위

□ 변수의 형태

종류	키워드	지속시간	정의되는 위치	유효범위
지역(자동)변수	(auto)	일시적	함수내부	지역적
전역(외부)변수		지속적	함수외부	전체적 모든 파일
정적변수	static	지속적	함수내부	지역적
정적외부변수	static	지속적	함수외부	전체적 특정 파일
레지스터변수	register	일시적	함수내부	지역적

12) 변수의 범위

□ 지역(자동)변수

- 함수 내에 정의되는 변수
- 초기화 되지 않으므로 임의의 값 (Garbage)가 저장된다.
- 사용하기 전에 반드시 초기화해야 한다.

- 함수가 실행되는 동안만 존재 한다.
- 변수가 선언된 함수에 의해서만 사용되어질 수 있다.

- 기본적으로 자동변수의 형태를 가지므로 앞에 auto 키워드를 추가할 수 있다.
- 함수의 인수로 쓰이는 변수도 역시 지역변수이다.

12) 변수의 범위

□ 전역(외부)변수

- 함수의 바깥에서 정의되는 변수, 보통 전역(Global)변수라고 한다.
- 범위는 전체 프로그램 내
- 프로그램이 실행되는 동안 메모리에 항상 배치되며 0으로 초기화 된다.
- 프로그램을 구성하는 대부분의 함수가 사용해야 하는 변수인 경우에만 사용하는 것이 좋다.

```
void funcA();  
int nGlobal;           // Global 변수  
void main(){  
    int nLocal;        // Local 변수  
    nLocal = 5;  
    nGlobal = 14;  
    funcA();  
}  
void funcA(){  
    nLocal = 5;         // error  
    nGlobal = 16;  
}
```

12) 변수의 범위

□ 정적 변수

- 자동변수를 정의할 때 static 키워드를 포함시켜 만든다
- 자동변수는 함수가 호출될 때마다 변수의 값을 보존하지 않으나 정적변수는 함수가 호출될 때 계속해서 값을 보존한다.
- 초기화 하지 않더라도 자동으로 초기화가 이루어진다.
- 함수 내부에 선언되면 그 함수에서만 사용할 수 있다.

□ 외부 정적 변수

- 외부 변수를 정의할 때 static 키워드를 포함시켜 만든다.
- 단지 변수가 정의된 파일 내에서만 사용할 수 있다.

12) 변수의 범위

```
#include<stdio.h>
void funA( );
void main( )
{
    for(int i=1; i<=5; i++)
        funA( );
}
void funA( )
{
    auto int a = 0;
    static int b = 0;
    a+=100;
    b+=100;
    printf("auto a = %d\n",a);
    printf("static b = %d\n",b);
}
```

12) 변수의 범위

□ 레지스터 변수

- CPU 에는 레지스터라는 몇 개의 저장영역이 존재하는데, 실제 데이터 연산은 레지스터에서 수행된다. 그런데 프로그램의 실행 속도를 증가 시키기 위해 레지스터에 변수의 값을 저장할 수 있다.
- 자동 변수를 정의할 때 `register` 키워드를 포함시킨다.
- 값을 저장하기 위한 공간이 남아 있지 않을 경우 일반적인 자동 변수처럼 취급

```
void funA();  
main()  
{  
    funA();  
    return 0;  
}  
void funcA(){  
    register int x;  
    ..  
}
```


1. OOP Concept

□ OOP Concept

1. 객체지향 언어 (Object Oriented Programming)의 특징
2. Class와 Object
3. 절차식(순차/구조) 프로그래밍과 객체지향 프로그래밍
4. 객체 지향적인 분석과 설계

1. OOP Concept

□ 객체지향 언어 (Object Oriented Programming)의 특징

○ 추상화 (Abstraction)

- 자세한 사항은 무시하고 본질적인 특징에 집중하도록 하는 과정
- 절차적 추상화 (Procedural Abstraction)
- 데이터 추상화 (Data Abstraction)

○ 캡슐화 (Encapsulation)

- Data 와 Method를 묶어 외부의 간섭 또는 잘못된 사용으로부터 보호하도록 하는 기능
- Data hiding

○ 상속성 (Inheritance)

- 한 Class 가 다른 Class 의 Data 와 Method 를 그대로 이어받을 수 있도록 하는 기능

○ 다형성 (Polymorphism)

- 하나의 interface로 다양한 행위들을 사용할 수 있도록 하는 기능
- C++ 에서는 중복함수, 템플릿, 가상함수 등을 통해 구현된다.

1. OOP Concept

□ Class 와 Object

- Class
 - Object 를 만들기 위한 자료형
 - Data 와 Method를 가진다.
- Object
 - 클래스의 구조에 따라 메모리를 할당 받은 변수
 - Class 에 의해 생성되어진 Instance

1. OOP Concept

□ 절차식 프로그래밍과 객체지향 프로그래밍

○ 절차식 프로그래밍 (Structured Programming)

- 함수중심으로 진행된다.
- Data가 노출된다.
- 재사용할 수가 없다.
- 프로그램이 복잡할수록 제어하기 어렵다.

○ 객체지향 프로그래밍 (Object Oriented Programming)

- Data 중심으로 진행된다.
- Data를 감춘다.
- 재사용할 수 있다.
- Data와 함수가 프로그램의 나머지 부분과 독립되어 제어하기 용이하다.

1. OOP Concept

□ 객체 지향 분석 및 설계

2. C++ is better than C

□ C++ is better than C

1. 함수 기본 매개변수(default parameter)
2. 함수 중복(function Overloading)
3. Inline 함수
4. C++의 참조 (Reference) 변수
5. 함수의 인수를 넘기는 방법
6. const
7. enum
8. new, delete

2. C++ is better than C

□ 함수 기본 매개변수(default parameter)

- 함수 호출에서 입력 인자를 빠뜨렸을 때 자동으로 사용될 값
- 원형선언에서 = 기호를 써서 만든다.
- 디폴트 인자를 지정할 때는 오른쪽 인자부터 지정해야 한다.

Ex)

```
void f1 ( int, int, int );  
void f2 ( int, int, int = 3 );  
void f3 ( int, int = 20, int );    // error  
void f4 ( int, int = 10, int = 4 );
```

2. C++ is better than C

□ 함수 중복(function Overloading)

- 같은 이름을 가진 함수를 2개 이상 정의해 쓸 수 있는 기능
- 입력인자의 개수가 서로 다르거나, 입력인자의 자료형이 서로 다르게 해서 사용
- 호출되는 함수를 구분하는 기준은 입력인자에 달려있다.

◆ 주의

입력인자는 같고 반환형만 다르게는 사용 불가능
함수 중복을 할 때 애매모호하게 사용하지 말 것

2. C++ is better than C

❑ 함수 중복 예제

```
#include <iostream.h>
int square ( int );
double square ( double );
int main()
{
    cout<<"Square value of 10: "<< square ( 10) << "\n";
    cout<<"Square value of 10.01: "<< square ( 10.01)<< "\n";
    return 0;
}
int square ( int n)
{
    return n*n;
}
double square ( double n)
{
    return n*n;
}
```

2. C++ is better than C

□ Inline 함수

○ 정의

- Program에서 함수에 대한 호출이 발생할 때, 함수를 호출하는 대신에 함수의 Code를 삽입

○ 장점

- 함수 호출과정의 Overhead를 줄임으로써 프로그램의 수행속도가 빠르다.
- 함수의 고유 기능을 사용할 수 있다.

○ 단점

- 반복해서 사용할 때는 실행파일을 크게 하는 문제점이 있다.

```
#include <iostream.h>
inline int square ( int );
int main()
{
    cout << "Square value of 10:" << square (10) << "\n";
    return 0;
}
int square ( int n){return n*n;}
```

2. C++ is better than C

□ 매크로 함수와 **Inline** 함수의 비교

매크로 함수	Inline 함수
#define 으로 정의한 함수	Inline 지정자를 가진 함수
preprocessor 에 의해 처리 Code 대치	Compile 시 코드 대치에 의한 방법으로 호출한 곳에 몸체를 갖다 둔다.
함수 호출에 따른 오버헤드가 없어 수행 속도가 빠르다	함수 호출에 따른 오버헤드가 없어 수행 속도가 빠르다
함수의 고유 기능을 사용할 수 없다.	함수의 고유 기능을 사용할 수 있다.
괄호를 과도하게 쓰지 않으면 잘못된 문자 대치가 일어날수 있다.	괄호를 과도하게 쓰지 않아도 문제가 없다.
반복해서 사용하면 실행파일 크기를 크게 한다.	반복해서 사용하면 실행파일 크기를 크게 한다.

2. C++ is better than C

□ Class 에서 inline member function으로 만드는 방법

1. Class의 선언에서 함수의 정의를 작성한다.
2. Member function정의 할 때 앞에 inline지정자를 추가한다.
(이 경우 inline member function은 Class 가 선언된 파일과 동일 파일 내에 존재해야 한다.)

```
class Rectangle{  
//1.  
private:  
    int m_nWidth, m_nHeight;  
public:  
    int GetArea (void)  
    {return m_nWidth*m_nHeight;}  
};
```

```
class Rectangle{  
//2.  
private:  
    int m_nWidth, m_nHeight;  
public:  
    int GetArea (void);  
};  
  
inline int Rectangle::GetArea(void)  
{return m_nWidth*m_nHeight;}
```

2. C++ is better than C

□ C++ 의 참조 (Reference) 변수

- 새로운 기억장소를 확보하는 것이 아니라 기존의 변수에 새로운 이름을 추가한다.
- 변수 : 메모리의 저장위치를 가리키는 것
- 레퍼런스 : 메모리의 위치에 대한 또 다른 이름
(반드시 초기화 되어야 함)
- Syntax

Data type & *reference_var* = *variable_name*

```
int i = 9;  
int &ri = i;
```

- 용도 : ① 함수의 입력인자형 ② 함수의 반환형 ③ 독립참조

2. C++ is better than C

□ 함수의 인수를 넘기는 방법

- Call by value
- Call by pointer
- Call by Reference

2. C++ is better than C

❑ const

- 상수를 정의할 때 사용한다.
- 절대로 변경되어질 수 없다.
- Syntax

const data_type const_var = initial_value

```
const float PI=3.14159f;  
float r;  
r = 2 * PI * 5;
```

- const pointer

2. C++ is better than C

□ enum

- 특정 값들의 집합으로 제한된 상수들을 만들 때 사용
- Syntax

enum Enum명 {*const_var* = initial_value, *const_var* = initial_value,.....};

```
enum Color{RED=0,BLUE=1,GREEN=2};  
cout << RED;  
Color c;  
C = BLUE;
```


2. C++ is better than C

□ new , delete

- 왜 Dynamic Memory 를 사용해야 하는가?
- new, delete 연산자
 - new를 사용해 자유기억공간에 메모리를 할당
 - delete를 사용해 포인터가 가리키던 메모리를 해제
- Syntax

Data_type * pointer_var = new **Data_Type**
delete **pointer_var**

```
int *ip= new int;  
delete ip;
```

- 주의점
 - **Memory leak**
 - **Dangling pointer**

3. Foundation of Class

□ Class

1. Class 와 Object
2. Class 작성단계
3. Class 선언
4. Class 정의
5. Object 생성 및 사용
6. 접근 지정자
7. This
8. 생성자, 소멸자
9. 생성자의 종류

3. Foundation of Class

□ Class 와 Object

- Class
Object 를 만들기 위한 자료형
Member Variable과 **Member Function**을 가진다.
- Object
클래스의 구조에 따라 메모리를 할당 받은 변수
Class 에 의해 생성되어진 **Instance**

3. Foundation of Class

□ Class 작성 단계

1. Class 선언
2. Class 정의
3. Object 생성

3. Foundation of Class

□ Class 선언

- Class 선언은 class keyword로 시작하고 Class 명을 쓰고, { } 안에 member를 나열한다.
- Member
 - Member variable : Class 의 속성을 나타내는 변수
초기값을 지정할 수 없다.
 - Member function : Class의 행동을 나타내는 함수
- Member 접근 지정자
 - Member를 접근하는데 일정한 제한을 가하는 것
 - Private, protected, public이 있다.
 - Member 접근 지정자 다음에는 : 를 써주어야 하고
일단 어느 하나로 지정하면 다른 것으로 지정할 때까지 앞의 것이 계속 유효하다.

3. Foundation of Class

□ Class 선언

```
class Rectangle{           // class keyword Class_Name
    public:                 // public keyword :
        void SetWidth ( int );
        long GetArea ( void );
        // return_type FunctionName ( parameters);

    private:
        int m_nHeight, m_nWidth;
        // data_type member_name;
};                          // 반드시 semicolon!!!
```

3. Foundation of Class

□ Class 정의

○ Member 함수의 정의

- Class의 행동을 나타내는 함수
- Member 함수의 몸체 정의에서는 Class명과 Class 소속을 나타내는 ::(scope resolution operator)를 함께 사용한다.

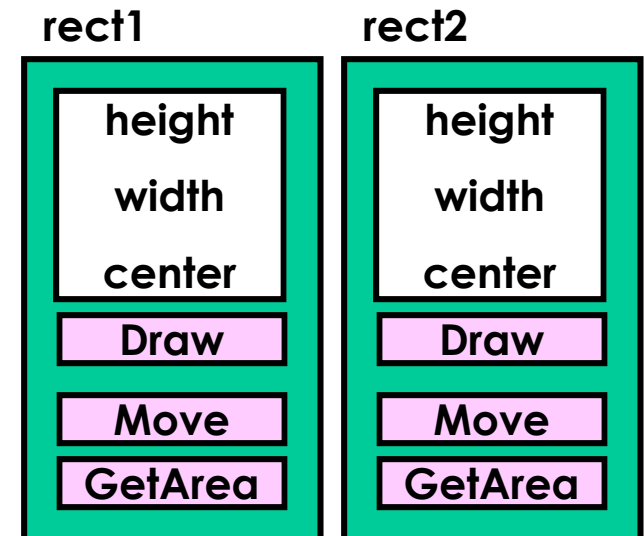
```
int Rectangle::GetArea (void)
{
    return m_nWidth*m_nHeight;
}
```

3. Foundation of Class

□ Object 생성

- 정수형 변수를 선언 하듯이 개체를 선언한다.
- Syntax
ClassName **objectvar_name**;
- Member 를 접근할 때에는 .(dot operator)를 이용한다.
- Syntax
obj.memfunction();

```
#include <iostream.h>
void main()
{
    Rectangle rect1;
    Rectangle rect2;
    rect1.setwidth(3);
    rect1.setHeight(10);
    int a;
    a = rect1.GetArea();
    cout << a << endl;
}
```



3. Foundation of Class

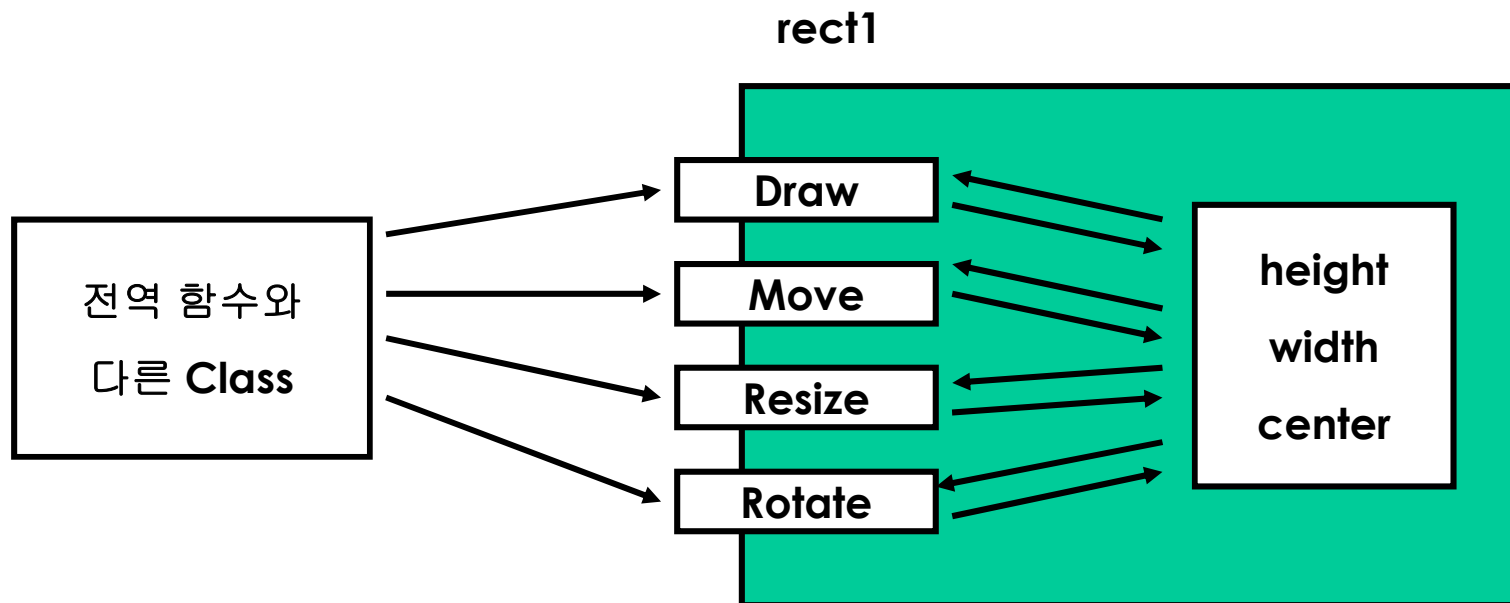
□ 접근 지정자

접근 지정자	Class 의 안에서 접근가능한가?	파생 Class 에서 접근가능한가?	Class 의 밖에서 접근가능한가?
public:	yes	yes	yes
protected:	yes	yes	no
private:	yes	no	no

- ◆ 접근 지정자를 생략하면 디폴트로 **private**이 된다.
- ◆ 일반적으로 **Member** 변수는 **private**으로 선언하고
Member 함수는 **public**으로 선언한다.

3. Foundation of Class

□ 접근 지정자의 사용



3. Foundation of Class

□ this pointer

- 호출한 객체를 가리키는 포인터
- 어느 Member 함수가 호출되든지 간에 자동으로 전달된다.
- 사용 : 자기 자신의 object 라는 것을 좀더 명확하게 하기 위해 자신의 object 를 반환할 때.

```
int Rectangle::GetArea(void)
{
    return this->m_nWidth * this->m_nHeight;
}
```

3. Foundation of Class

□ 생성자와 소멸자

생성자	구분	소멸자
Class 명과 동일	명칭	Class 명 앞에 ~를 붙임
Object 생성 시	호출	Object 소멸 시
Object 초기화	용도	Object 정리
어떤 인자도 가능	인자	인자 불가능
중복 가능	중복	중복 불가능
지정할 수 없다	반환	지정할 수 없다

3. Foundation of Class

□ 생성자 함수의 종류

- 기본 생성자 (default constructor)
- 인자 생성자 (argument constructor)
- 복사 생성자 (copy constructor)

3. Foundation of Class

□ 생성자 (Constructor), 소멸자(Destructor)

```
class Rectangle{  
private:  
    int m_nWidth, m_nHeight;  
  
public:  
    // Default Constructor  
    Rectangle();  
    // Argument Constructor  
    Rectangle( int , int );  
    // Destructor  
    ~Rectangle ();  
};
```

```
Rectangle::Rectangle()  
{  
    m_nWidth = 0;  
    m_nHeight = 0;  
}  
  
Rectangle::Rectangle(int w, int h)  
{  
    m_nWidth = w;  
    m_nHeight = h;  
}  
  
Rectangle::~~Rectangle()  
{  
    cout << "Rectangle 소멸자  
    ...\\n";  
}
```

```
int main()  
{  
    Rectangle rect1;  
    Rectangle rect2;  
    Rectangle rect3(10,10);  
    return 0;  
}
```

4. Advance of Class

❑ Advanced..

1. Const member function
2. Const object
3. Colon initialization
4. Operator overloading
5. Friend function
6. Static member

4. Advance of Class

❑ Const Member Function

- Member variable 의 값을 변경하지 않겠다는 것을 약속해주는 함수
- Const member function에서는 member variable 의 값을 바꿀 수도 없으며 Const member function이 아닌 다른 member function을 호출할 수도 없다.

```
class Rectangle{  
private:  
    int m_nWidth, m_nHeight;  
  
public:  
    int GetArea (void) const;  
};
```

```
int Rectangle::GetArea(void) const  
{  
    //m_nWidth = 0; error  
    return m_nWidth*m_nHeight;  
}
```


4. Advance of Class

❑ Const Object

- Object의 Member variable 의 값을 바꾸지 않겠다는 것을 약속해준다.
- Const object 자체가 l-value 로 사용될 수 없으며
Const member function만 call 할 수 있다.

```
class Rectangle{  
private:  
    int m_nWidth, m_nHeight;  
  
public:  
    void SetWidth(int w)  
    {m_nWidth = w;}  
    int GetWidth() const  
    {return m_nWidth;}  
    int GetArea (void) const  
    {return m_nWidth*m_nHeight;}  
};
```

```
int main()  
{  
    const Rectangle r1;  
    Rectangle r2;  
    //r1= r2 error  
    //r1은 const object  
    int r;  
    r=r1.GetArea();  
    //r1.SetWidth (100); error  
    //SetWidth는 const member 가 아님  
    return 0;  
}
```

4. Advance of Class

□ Colon 초기화

- Member 변수를 초기화하는 다른 문법으로써
{} 이 시작되기 전에 : 을 쓰고 나서 Member 변수를 초기화 시킨다.
- = 은 사용할 수 없고 반드시 () 를 사용해야 한다.
- 생성자에서만 사용할 수 있다.

```
Rectangle::Rectangle() : m_nWidth (0), m_nHeight (0)
{
}

Rectangle::Rectangle(int w, int h) : m_nWidth (w), m_nHeight (h)
{
}
```

5. Inheritance

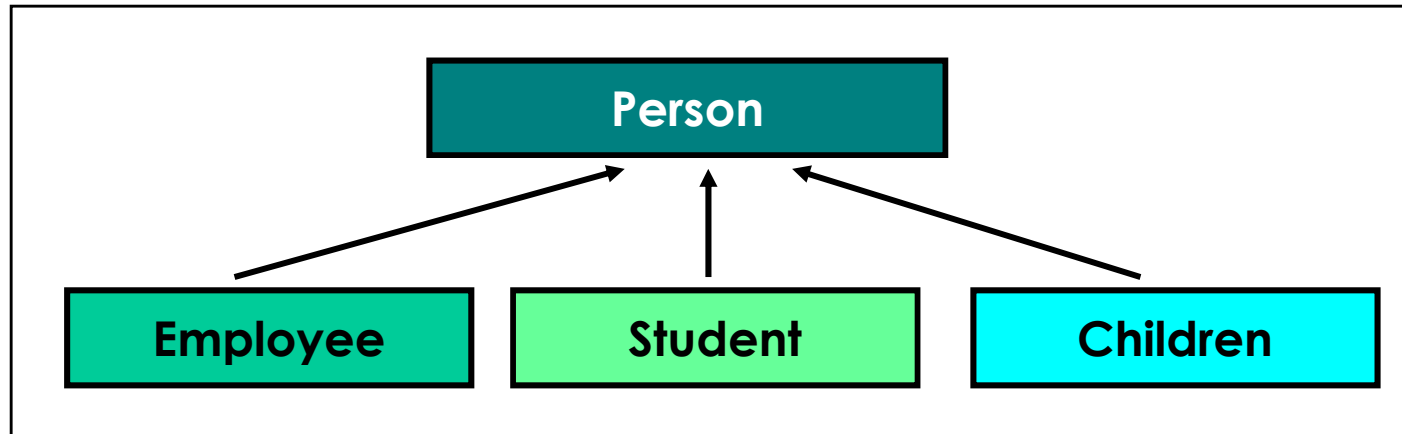
□ Inheritance

1. Inheritance와 Embedding
2. Inheritance
3. Member 접근 변경자
4. 상속되는 것과 상속되지 않는 것
5. Derived class 의 member function
6. Inheritance와 생성자가 호출되는 순서
7. Inheritance와 소멸자가 호출되는 순서
8. 다중상속

5. Inheritance

□ 상속성(Inheritance)

- 기존에 정의되어 있는 Class로부터 Data와 Method를 이어받아 새로운 Class 를 만드는 기능
- 상속관계에서 상속해주는 Class 를 **Base Class** (Parent Class)라 하고, 상속 받은 Class 를 **Derived Class** (Child Class)라 한다.
- <A **is a** B> 또는 <A **is kind of** B> 의 관계가 성립될 때 B는 Base class, A는 Derived Class 로 만들 수 있다.



5. Inheritance

□ 상속Class 선언

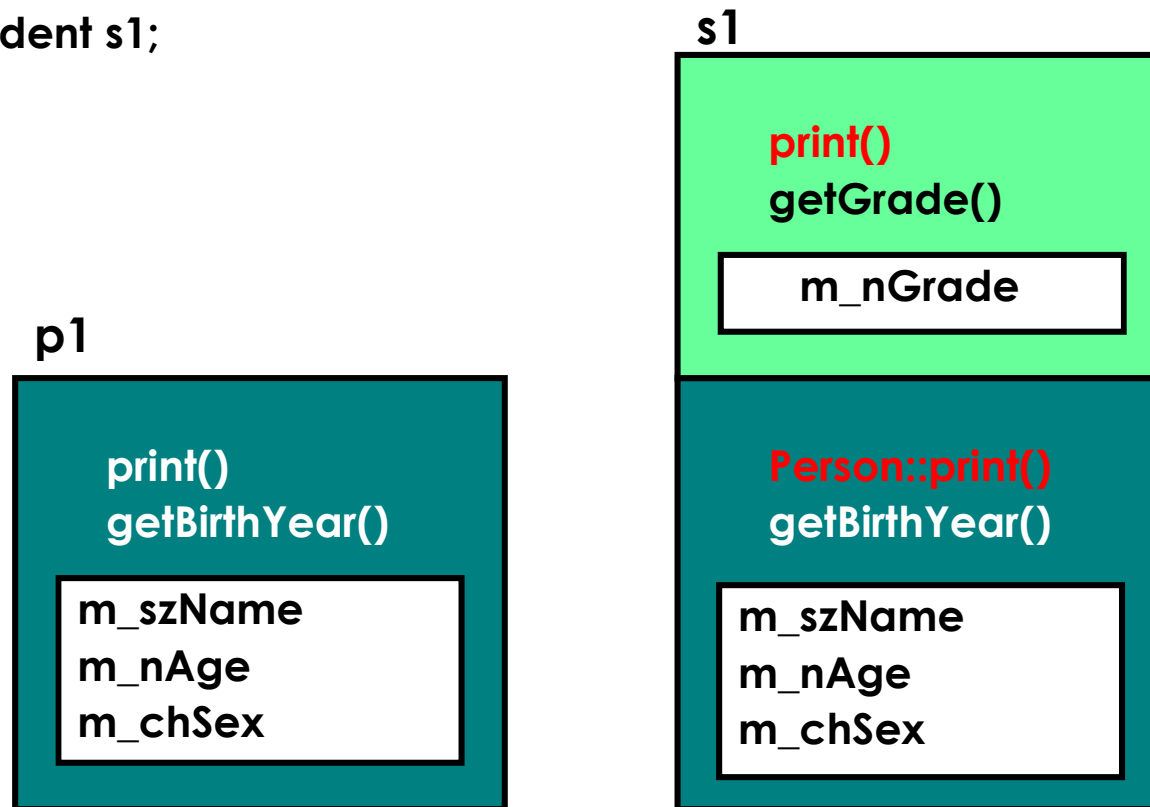
```
class Person{
private:
    char m_szName[10];  char m_chSex;  int m_nAge;
public:
    Person();
    ~Person();
    int getBirthYear();
    void print();
};

class Student : public Person {
    // class-keyword class_name : 접근변경자-keyword Base_class_name
private:
    int m_nGrade;
public:
    Student();
    ~Student();
    void print();
    int getGrade();
};
```

5. Inheritance

□ Object 생성

Person p1;
Student s1;



5. Inheritance

□ Member 접근 변경자

- 상속될 때 Base class 내 Member 의 Scope를 변경
- public, protected, private 가운데 하나를 사용할 수 있다.
- 그러나 95% 이상 public 이 사용된다.

접근 변경자	private member	protected member	public member
private	private	private	private
protected	private	protected	protected
public	private	protected	public

5. Inheritance

❑ 상속되어지는 것과 상속되어지지 않는 것

○ What is Inheritance?

Data Members

Most Member Functions

○ What is **not** Inheritance?

Constructors

Destructors

Friend Function

5. Inheritance

□ Derived Class의 Member

- 그대로 이어받아 사용하는 Member

Base class 의 Member 변수와 Member 함수를 Derived class에서 그대로 이어받아 사용

Ex) Person class 의 **getBirthYear()** Member함수

- 재정의(override)한 Member

Base class에 있던 Member 함수와 원형이 같은 Member 함수를 Derived Class에서 재정의(Override)해서 사용

Ex) Person class 의 **print()** Member 함수를 Student class 에서 재정의

- 새로 추가한 Member

Base class 에 없던 Member 변수와 Member 함수를 Derived class에서 새로 추가해 사용할 수 있다.

Ex) Student class의 **getGrade()** Member 함수

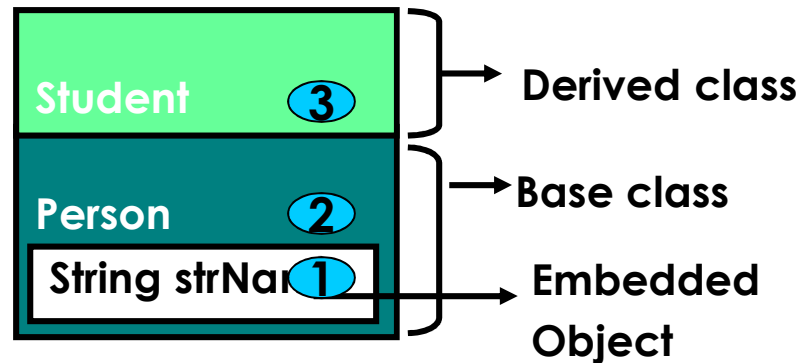
5. Inheritance

□ 상속과 생성자가 호출되는 순서

○ 원칙

1. Base Class
 1. Embedded Object, if any
 2. Surrounding
2. Derived Class
 1. Embedded Object, if any
 2. Surrounding

Student Object



```
String::String(char * s)
{
    ....
}
Person::Person(char * s, int a, char ch) : String ( s )
{
    ....
}
Student::Student(char * s, int a, char ch, int g)
: Person( s, a, ch ) m_nGrade ( g )
{
    ....
}
Student std1( "OOP", 10, 'M', 100);
```

Arrows indicate the construction order: 1 (String strName) is constructed first, then 2 (Person), then 3 (Student), and finally 4 (String constructor) is called for the first parameter.

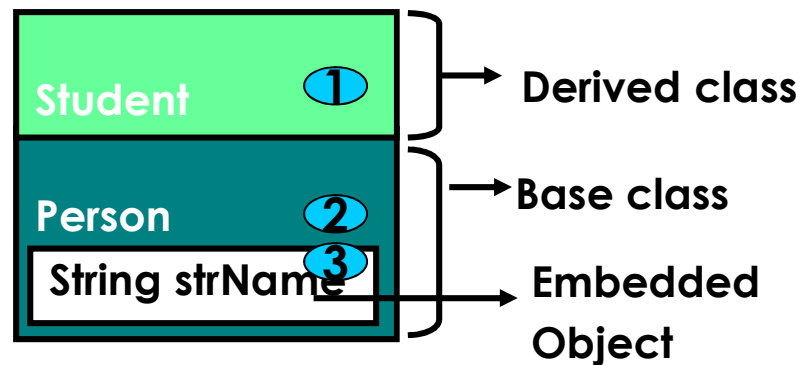
5. Inheritance

□ 상속과 소멸자가 호출되는 순서

○ 원칙

1. Derived Class
 1. Surrounding
 2. Embedded Object, if any
2. Base Class
 1. Surrounding
 2. Embedded Object, if any

Student Object



6. Polymorphism

□ Polymorphism

- Class 형 변환
- Virtual function
- Pure Virtual function
- Abstract class

6. Polymorphism

□ Class 형 변환

- Base Class 에 대한 포인터나 레퍼런스 변수는 Derived Class 의 Object 를 가리킬 수 있다.
- 그러나 반대로는 사용할 수 없다.
- 이 경우 이 레퍼런스 변수를 통해 override한 Member 함수를 호출하면 Base Class 로부터 상속 받은 Member 함수가 호출된다.

```
Person objPs;  
Student objStd;
```

```
Person &r1 = objPs;  
Person &r2 = objStd;
```

```
r1.print ();    //Person::print()가 호출  
r2.print ();    //Person::print()가 호출
```

6. Polymorphism

□ 가상 함수 (Virtual Function)

- 코드의 주소가 Compile시에 결정되어 있지 않다가 실행 시에 결정되는 함수
- Base class 에서 virtual 지정자를 사용해 가상 함수를 만든다.
Derived class 에는 생략해도 자동으로 가상함수가 된다.
- Base class 의 레퍼런스 변수가 Derived class 의 object 를 가리키고 있을 때 이 레퍼런스 변수를 통해 가상 함수를 호출하면 Derived class 의 override한 Member 함수가 호출된다.

6. Polymorphism

□ 가상함수 예제

```
class Person{
private:
    //...
public:
    virtual void print();
    //...
};
class Student : public Person {
private:
    //...
public:
    virtual void print();
    // virtual 생략가능
    //...
};
```

```
int main()
{
    Person objPs;
    Student objStd;

    Person &r1 = objPs;
    Person &r2 = objStd;

    r1.print ();
    //Person::print()가 호출
    r2.print ();
    //Student::print()가 호출

    return 0;
}
```

6. Polymorphism

□ 순수 가상 함수 (Pure Virtual Function)

- 몸체정의를 갖지 않고 원형 선언만으로 구성된 함수
- 순수 가상 함수를 명시 하는 방법은 원형 선언 시 끝에 **=0** 를 넣어준다.

□ 추상 Class (Abstract class)

- 순수 가상 함수를 하나 이상 포함한 함수
- 그 자체의 Object 를 만들어 사용하지는 못한다.
- Derived Class 는 반드시 순수 가상 함수를 override해야 한다.
만약 override하지 않으면 derived class 역시 추상 class 가 된다.
- 라이브러리 설계자가 미리 프로그램의 뼈대를 구성해 놓기 위해 필요하다.

```
// 추상 class
class GeoShape{
public:
    virtual void print() = 0;
    //순수 가상 함수
    //...
};
```

```
int main()
{
    GeoShape obj;           // error
    return 0;
}
```


7. Programming

□ 상용 프로그램 제작 요건

- 프로그래밍 언어 지식
 - 언어 문법, 개발도구(Visual C++ > Win32 SDK)사용법
 - 프로그래밍 스타일(ex. OOP)
- 라이브러리 지식
 - Class Library(MFC), 자료구조, 각종 API(ex. Win32 API)
- 개발환경 지식
 - 운영체제, 하드웨어
- 문제영역에 대한 지식(ex. 네트워크, 데이터베이스)

□ JAVA

- JAVA = 프로그래밍 언어 + 라이브러리 + 운영체제(플랫폼)

8. Windows Programming

❑ Windows Programming

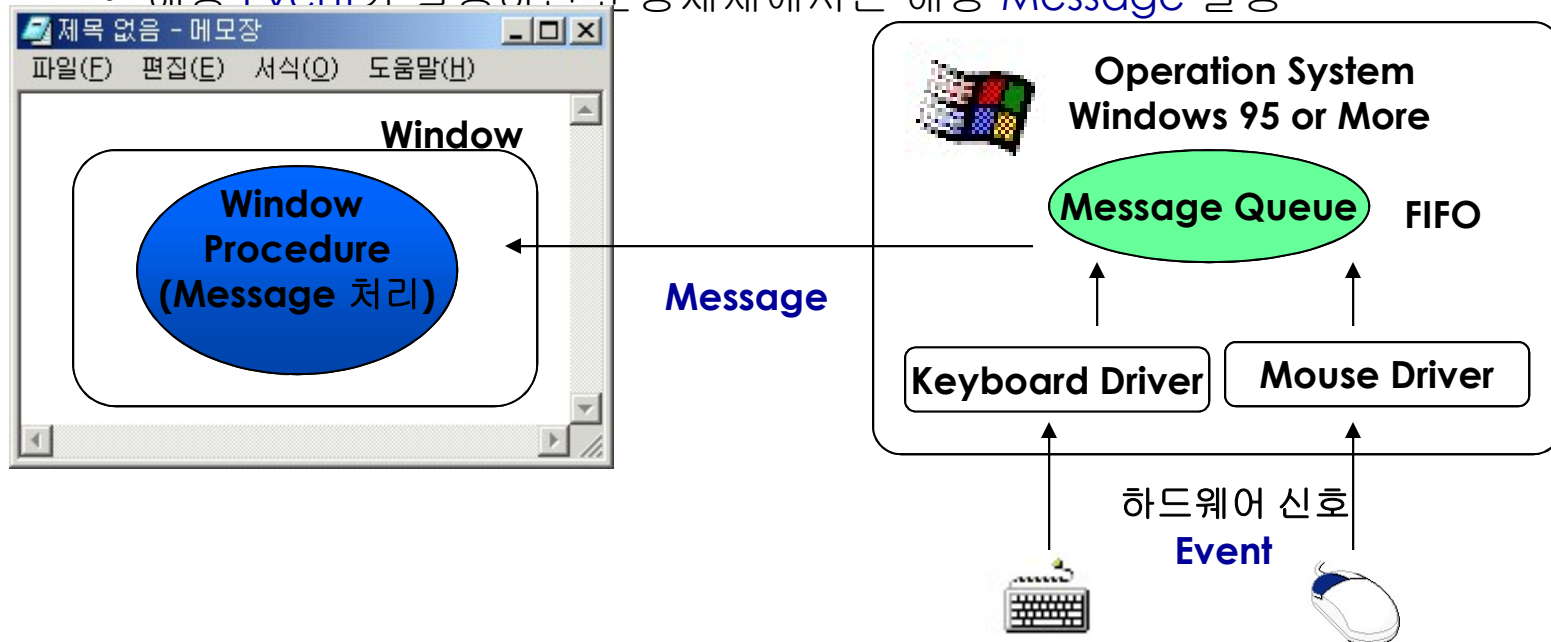
1. Windows Programming
2. Event & Message
3. Microsoft Windows Operating System 구성
4. Microsoft Windows Operating System 그래픽 출력 특징
5. DC(Device Context)

8. Windows Programming

□ Windows Programming

○ Event-Driven(이벤트 처리) 방식

- 이벤트가 발생할 때 프로그램은 어떻게 해야 한다고 정의
- 해당 **Event**가 발생하면 운영체제에서는 해당 **Message** 발생



○ Message

- 해당 이벤트 발생에 대해 메시지로 만들어 내는 일은 운영체제가 담당

8. Windows Programming

□ Event & Message

- 정수형 변수며 상수로 사용(windows.h 참조: 200여개)
- WM_*: General Window
 - WM_CREATE: 윈도우가 생성되는 순간에 발생하는 메시지
 - WM_MOUSEMOVE: 어떤 윈도우 위로 마우스 움직임
 - WM_LBUTTONDOWN, WM_LBUTTONUP, WM_LBUTTONCLK
 - : 어떤 윈도우 위로 Mouse 왼쪽 버튼 ***
 - WM_CHAR, WM_SYSCHAR, WM_KEYDOWN, WM_SYSKEYDOWN
 - : 어떤 윈도우 위로 Keyboard ***
- BM_*: Button Control
- CDM_*: Common Dialog Box
- DBT_*: Device
- DL_*: Drag List box
- PBM_*: Progress Bar
- SB_*: Status Bar
- TB_*: Tool Bar
- ...

8. Windows Programming

□ Microsoft Windows Operation System 구성

- 윈도우 운영체제는 다음과 같은 세 가지의 Dynamic Library로 구성
Win32 API함수의 대부분이 이 세 가지 DLL에 의해 제공됩니다.
 - Kernel: 메모리와 하드웨어를 관리
 - User: 사용자 인터페이스와 윈도우를 관리
 - GDI(Graphic Device Interface): 화면처리와 그래픽을 관리

8. Windows Programming

□ Microsoft Windows Operating System 그래픽 출력 특징

- 화면 사용 영역의 제약/좌표 계산 방법
 - 도스: 프로그램이 시스템 자원 독점하기 때문에 화면 출력 관계 없음.
 - 윈도우 운영체제: 반드시 자신이 차지하는 영역에만 출력(멀티태스킹)
- 출력 장치에 영향을 받지 않는다.
 - 윈도우 운영체제는 어떤 프린터/그래픽 카드가 장착되어도 똑같은 출력 함수를 사용할 수 있음. 이 장치 독립성 메커니즘은 디바이스 드라이버에 의해 구현되며 장치가 바뀌면 다른 디바이스 드라이버만 바꿈으로써 함수는 영향을 받지 않는다.
- 이러한 윈도우 운영체제의
그래픽 출력에 대한 모든 특징은 DC(Device Context)가 담당

8. Windows Programming

□ DC(Device Context)

- 화면 출력에 필요한 모든 정보를 갖고 있는 데이터 구조체

□ DC를 이용한 프로그래밍 절차

1. 출력을 수행할 DC 생성
2. 이전에 사용된 DC 상태 보존
3. Font(글자), Pen(그림의 윤곽선), Brush, (폐곡선의 내부), Bitmap(영상)등 출력에 사용할 도구 생성 후 이를 DC에 연결
4. 윈도우 배경색, 전경색 등의 정보를 DC에 설정
5. 출력을 수행
6. 출력 후 사용했던 도구 소멸
7. 이전 DC 상태를 복구
8. 생성했던 DC를 소멸

9. Windows Programming- Sample

□ Sample

1. Win32 API + C 로 만들어진 “Hello, Windows”
2. Win32 API + C 로 만들어진 Window 입출력 프로그래밍
3. Keyboard
4. Mouse
5. Timer
6. Win32 API Wizard
7. Win32 API + C++ 로 만들어진 “Hello, Windows”::OOP
8. MFC AppWizard

9. Windows Programming- Sample

□ Win32 API + C로 만들어진 “Hello, Windows”

○ 응용 분야

- Windows Programming의 개념 이해
 - RAD 툴에 비해 Win32 API 프로그래밍은 Windows Programming의 개념을 그대로 코드화 하고 있다.
- Win32 API 사용법
- Window Message 원리와 사용
- Window Procedure 원리와 사용

○ Visual C++ → File → New → Win32 Application

- Project Name: HelloWin
- Sept 1 of 1: An empty project

○ File → New → C++ Source File

- Add to project
- File Name: HelloWin.c

○ Build → Build HelloWin.exe or <F7>

○ Add

- Using WM_LBUTTONDOWN, MessageBox(), Beep()

○ Reference: MSDN/Platform SDK/Win32 API/Overview of the Win32 API

9. Windows Programming Sample

□ Win32 API + C 로 만들어진 Windows 입출력 프로그래밍

○ 응용 분야

- Windows 입출력 Programming의 개념 이해
- GDI(Graphic Device Interface), DC(Device Context) 개념
- Timer, Keyboard, Mouse 메시지 처리

○ Visual C++ → File → New → Win32 Application

- Project Name: WinIO
- Sept 1 of 1: An empty project

○ File → New → C++ Source File

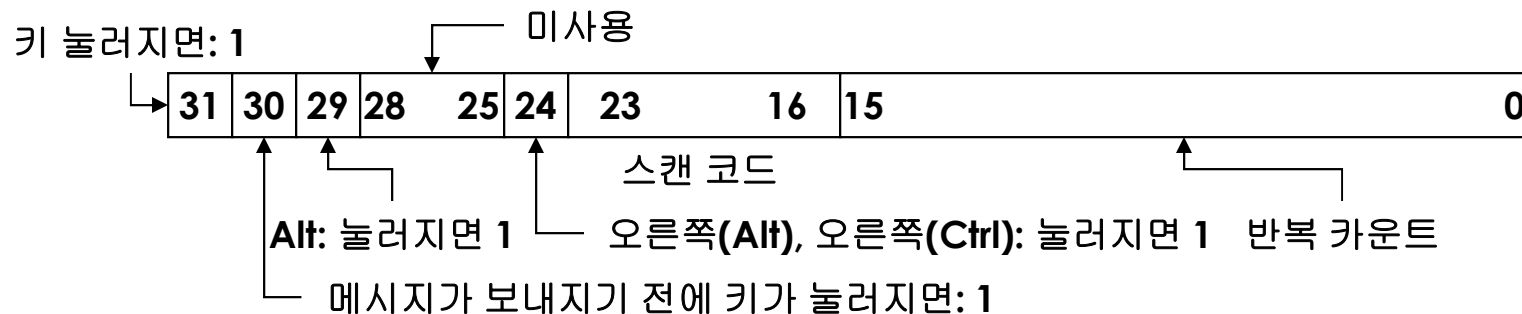
- Add to project
- File Name: WinIO.c

○ Build → Build WinIO.exe or <F7>

9. Windows Programming- Sample

□ Keyboard

- 입력된 문자(알파벳, 숫자, 기호)의 ASCII Code를 wParam으로 전달
: WM_CHAR 메시지 처리
- 특수키(Ins, Home, 방향키 ...)의 Virtual Key Code를 wParam으로 전달,
: WM_KEYDOWN 메시지 처리
- Virtual Key Code(VK_*): MSDN 참조(Ex: VB_BACK/08/Backspace)
- wParam/lParam: 메시지와 관련된 정보, 각 32bit
 - w: WORD- 16bit(16Bit 운영체제의 잔재)
 - l: LONG- 32bit
- lParam 구조



9. Windows Programming- Sample

□ Mouse

○ 마우스 메시지

- lParam의 상위 WORD: 마우스 클릭 y좌표- HIWORD(lParam) 사용
- lParam의 하위 WORD: 마우스 클릭 x좌표- LOWORD(lParam) 사용
- wParam: 마우스 버튼, Shift, Ctrl키의 상태 정보
 - MK_CONTROL
 - MK_LBUTTON, MK_MBUTTON, MK_RBUTTON
 - MK_SHIFT

9. Windows Programming- Sample

□ Timer

○ 응용 분야

- Timer: WM_TIMER 메시지
- Visual C++ → File → New → Win32 Application
- Project Name: WinTimer
- Sept 1 of 1: An empty project

○ File → New → C++ Source File

- Add to project
- File Name: WinTimer.c

○ Build → Build WinTimer.exe or <F7>

9. Windows Programming- Sample

□ Win32 API Wizard

- 응용 분야
 - Win32 API Wizard 기능 이해
 - C/C++를 이용한 Win32 Windows Application 변형
- Visual C++ → File → New → Win32 Application
 - Project Name: HelloWin2
 - Sept 1 of 1: A typical "Hello World!" Application
- Build → Build HelloWin2.exe or <F7>

9. Windows Programming- Sample

□ Win32 API + C++ 로 만들어진 “Hello,Windows”::OOP(1)

○ 응용 분야

- 재사용이 가능한 클래스 라이브러리 구축: OOP 사용

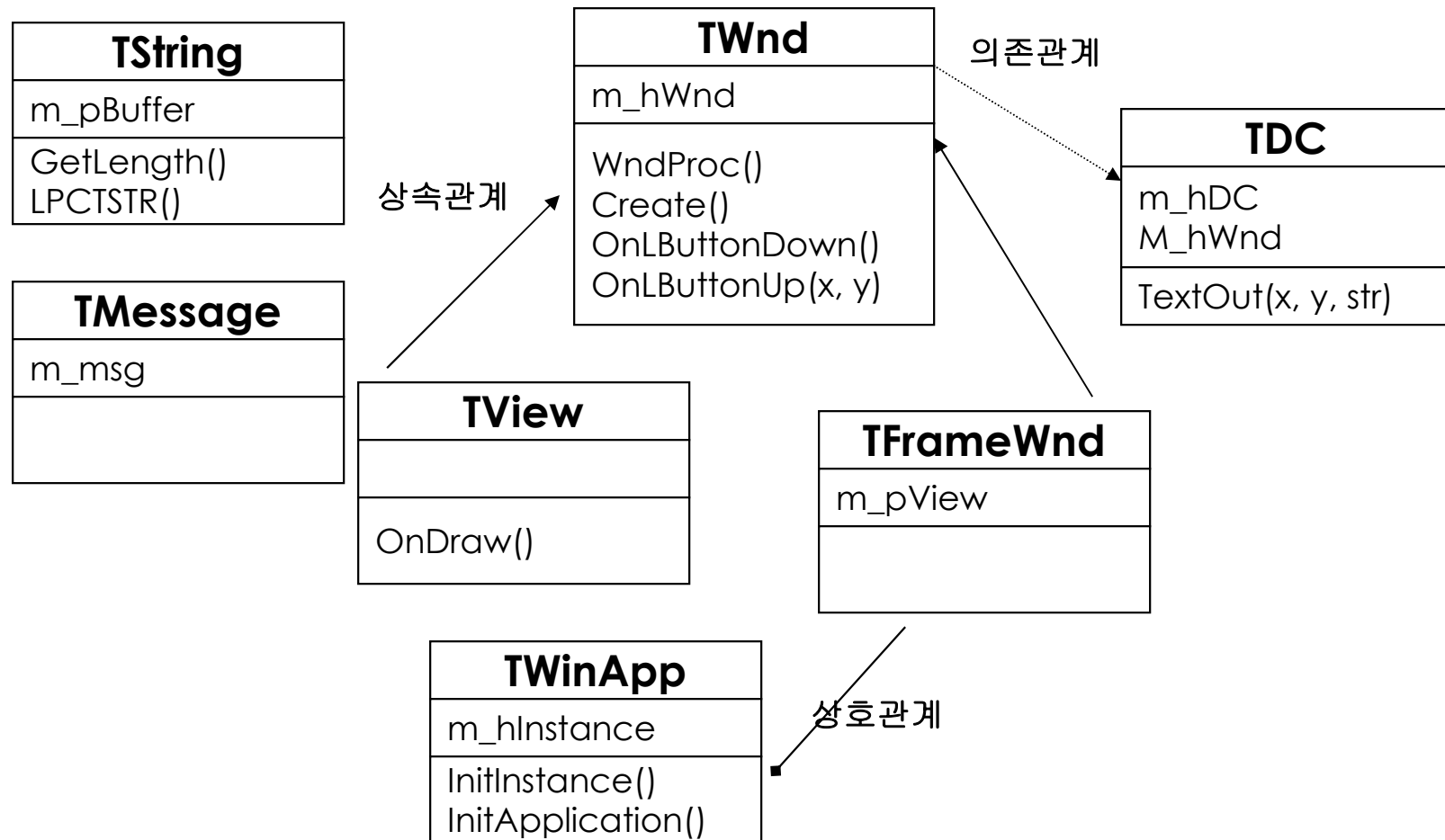
○ 객체지향 분석/설계(문제 이해 > 클래스로 만들 부분 식별 > 클래스 역할 지정 > 클래스 관계 식별 > 클래스 구현)에 기초한 Framework for Application Programming에서 사용할 주요 클래스(cf. MFC)

주요개념	설 명	클래스 이름
Windows Application	Win32 어플리케이션	TWinApp
Window	화면에 표시되는 사각영역/사용자와 상호작용	TWnd
Frame Window	기본 틀이 되는 메인 윈도우	TFrameWnd
Client Area	데이터를 표시하는 부분	TView
Device Context	출력을 위한 정보	TDC
Message	프로그램에 전달되는 이벤트 정보	TMessage
String	문자열	TString

9. Windows Programming- Sample

□ Win32 API + C++ 로 만들어진 “Hello,Windows”::OOP(1)

○ UML에 기초한 객체 관계 식별



9. Windows Programming- Sample

❑ MFC(Microsoft Foundation Class)

- C++ Class Library
- Message Map: MFC에서는 메시지를 Win32 SDK에서처럼 윈도우 프로시저에서 처리하는 것이 아니라 메시지 맵(Message Map)을 사용

❑ MFC AppWizard

- 응용분야
 - MFC 개념 및 사용법
- Visual C++ → File → New → MFC AppWizard(exe)
 - Project Name:
 - Sept 1 of 1:
- Build → Build 3DObject.exe or <F7>

10. .NET Framework

□ .NET Windows Programming

○ .NET Framework

- Microsoft 차세대 플랫폼
- C#, Visual C++ .NET, Visual Basic .NET, Jscript등의 모든 언어가 하나의 Runtime Library에서 실행되는 환경
- 현재 Visual Studio 6.0로 개발된 모든 프로그램은 그대로 운영체제의 Win32 API를 직접 호출해 사용할 수 있음.
- .NET 언어로 개발된 프로그램은 .NET Framework를 통해 Win32 API에 호출

